

# ORION GCS

Ground control station for UGVs
on ArduPilot Rover and ESP32 boards

## User manual

Program version 0.9.3 · Author: Anton Vasylenko · © 2026

# Contents

1. Installation and updates
2. Window layout
3. Connecting the rover (“Link”)
4. The “Drive” screen
5. Map, coordinates, ruler and routes
6. Readiness to drive
7. Trip recording, snapshots and .tlog
8. “3D view” and bench check
9. Radio: receiver on the rover (ELRS / CRSF)
10. Radio over USB: driving from the PC
11. Accelerometer and compass calibration
12. Cameras
13. Parameters
14. ESP32 board: firmware builder
15. Forwarding to Mission Planner / QGroundControl
16. Starlink
17. UGV Carrier v2 board
18. MAVLink signing: protecting the link
19. Access from other devices and multiple operators
20. System: dependencies, updates
21. Troubleshooting

**How to read this manual.** The screenshots carry orange circles with numbers — under each screenshot the same number explains what is set up at that spot. The screenshots were taken with the built-in simulator, so the map in them is empty (no satellite imagery) and the camera shows a test signal.

The program has two interface languages: Ukrainian and English. The **УКР / EN** switch is in the top right corner; all button names in this manual are given in English.

# 1. Installation and updates

## What you need

- A PC with Windows 10 or 11 (64-bit) or 64-bit Linux — requirements below.
- A rover with an ArduPilot Rover flight controller, an ESP32 board with ORION firmware or a UGV Carrier v2 board.
- A link: a USB cable to the flight controller, a telemetry radio modem, ESP32 Wi-Fi or a network (Starlink/VPN).
- Optional: a Radiomaster transmitter (EdgeTX/ExpressLRS), IP cameras.

## PC requirements

| | **Minimum**<br>**telemetry, map, one H.264 camera** | **Recommended**<br>**3 cameras, recording, 3D view, several vehicles** |
|---|---|---|
| System | Windows 10 or 11 (64-bit) or 64-bit Linux. Windows 7 and 8 are not supported. | |
| CPU | 2 cores (6th-gen Intel Core i3, Celeron N5105 or similar) | 4 cores (8th-gen Core i5 or newer, Ryzen 3/5) |
| Memory | 4 GB (the program uses ≈300–500 MB) | 8 GB |
| Disk | ≈500 MB for the program + room for recordings (one 1080p camera ≈1–2 GB per hour) | SSD |
| Screen | 1366×768 | 1920×1080 |
| Graphics | Any with hardware H.264 decoding: integrated Intel HD 500 and newer, AMD or NVIDIA from the last ≈8 years. | |

**What loads the PC most:** the “Transcode” checkbox for cameras (H.265 / MJPEG → H.264) and recording with telemetry in the frame — each such camera takes 1–2 CPU cores, so it is better to set H.264 in the camera itself; three 1080p cameras at once — weak integrated graphics may not keep up; the 3D view (loads the graphics card only on its own page). The MAVLink link, .tlog recording, forwarding to Mission Planner, video recording without transcoding and WebRTC via go2rtc barely load the PC.

> For a weak device: run ORION on one PC and open the interface in a browser on a tablet or laptop on the same network at `http://PC-IP:8765` — the main work is done on the PC and the tablet only shows the screen. Signing in from other devices requires a password (section 19).

## Installation

1. Open github.com/megagoldenmen-bot/orion-releases/releases/latest and download `ORION-GCS-Setup-0.9.3.exe`.
2. Run the file. If Windows shows “Windows protected your PC”, click **“More info” → “Run anyway”** (the installer has no paid code-signing certificate).
3. Choose the installation folder (or keep the default) and press “Install”. The installer asks for administrator rights — they are needed to add a Windows Firewall rule for incoming telemetry (UDP 14550 and so on).
4. Start the program with the **ORION GCS** shortcut on the desktop or from the Start menu.

> Everything needed is already inside the installer: Node.js, the serial port driver and ffmpeg for video. No need to install them separately. The ESP32 firmware build tools (≈600 MB) are downloaded only on demand — in the “System” or “ESP32 board” section.

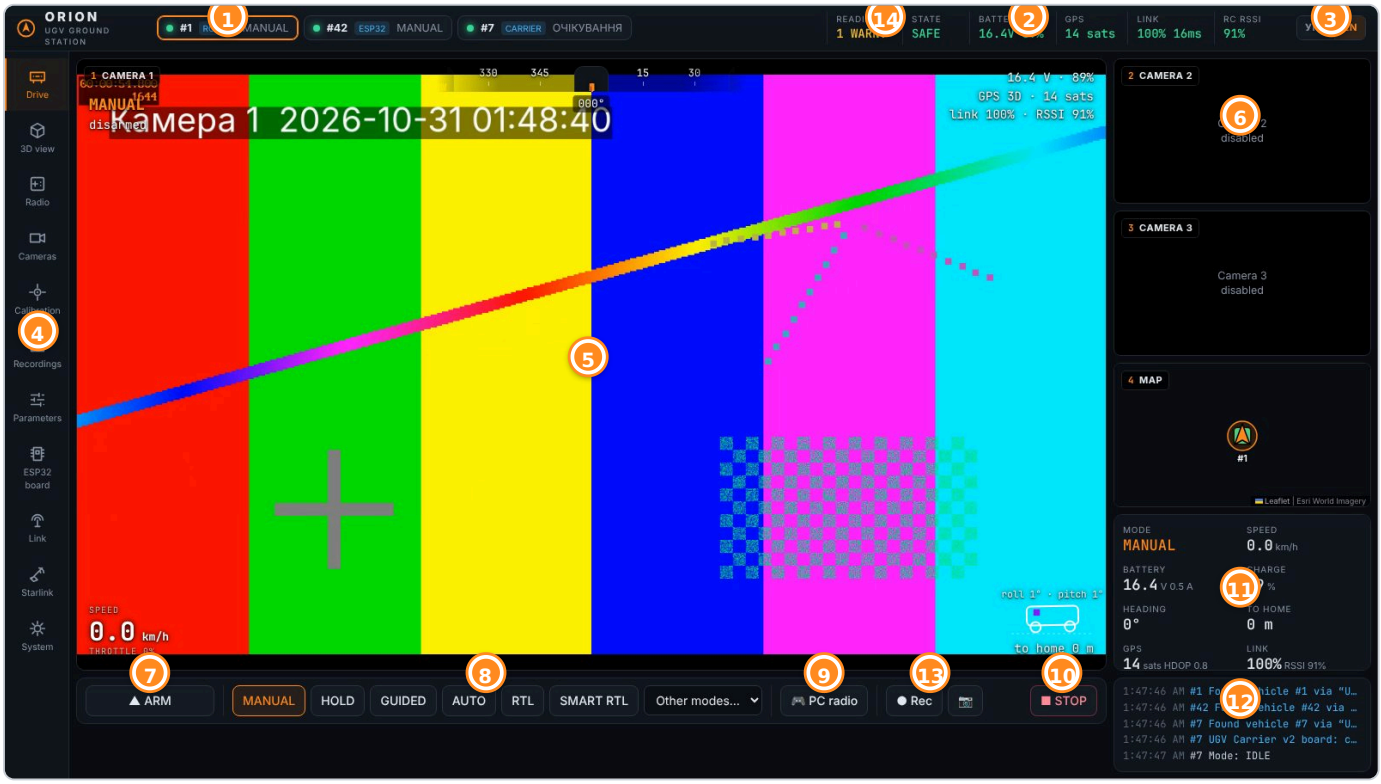
## Linux

For Linux there is `ORION-GCS-0.9.3.AppImage` on the same page. Make it executable (`chmod +x`) and run it. Install ffmpeg from your distribution's repository (`sudo apt install ffmpeg`); for a flight controller over USB, add your user to the `dialout` group. The AppImage updates the same way as the Windows version.

## Updates

On start-up the program checks for a new version by itself. If there is one, a message appears with the list of changes and an update button: the program downloads the new version, closes and restarts already updated. To check manually, go to **“System” → “Program updates”**. Settings, radio and board profiles are kept across updates.

# 2. Window layout

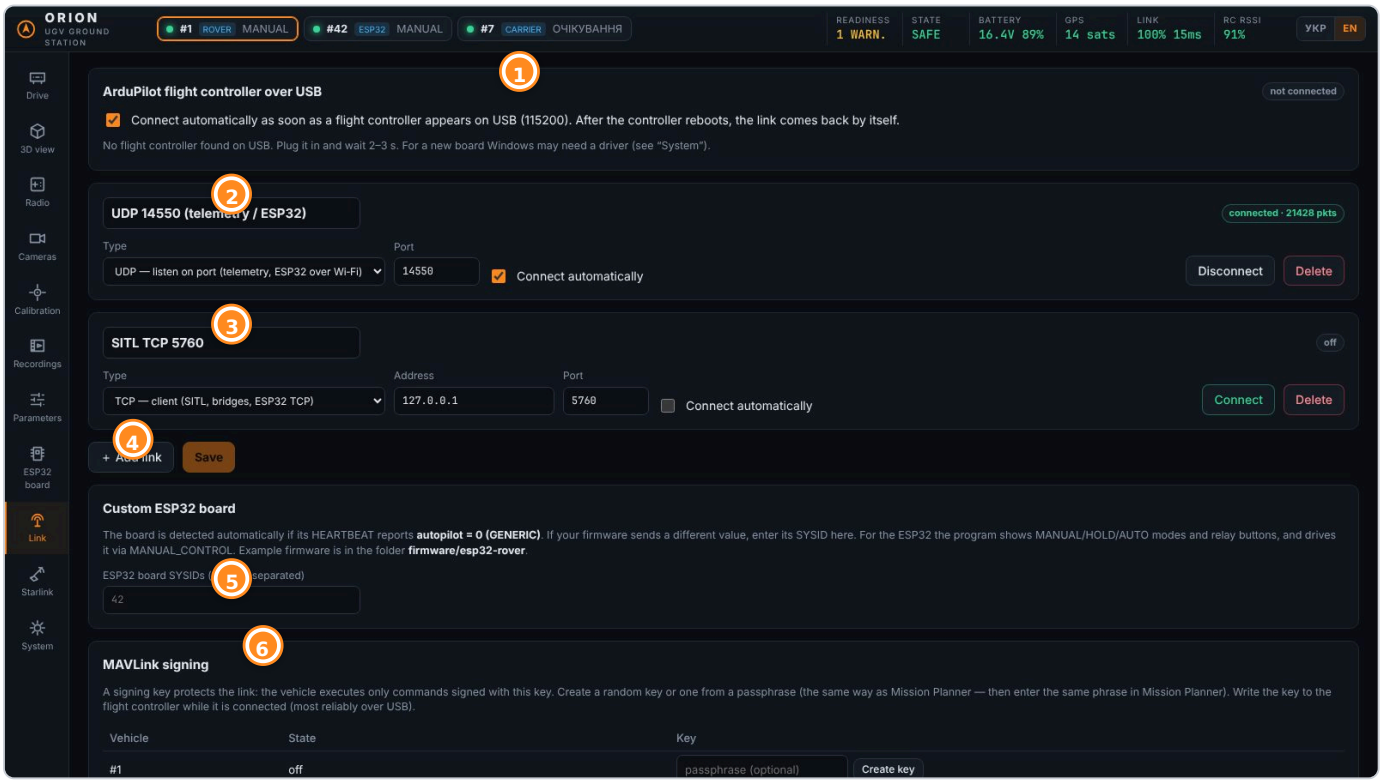



*Main window (“Drive” section)*

1. **Vehicles.** Every detected rover is a separate chip: number (SYSID), type (ROVER — ArduPilot, ESP32 — custom board) and current mode. A click selects the active vehicle — all sections work with it.
2. **Active vehicle state:** safety (SAFE / ARMED), battery voltage and charge, number of GPS satellites, link quality, transmitter RSSI.
3. **Interface language** — УКР / EN.
4. **Side menu of sections.**
5. **Large view** with HUD: heading, speed, throttle, mode, time, voltage, GPS, link, roll and pitch, distance home.
6. **Small views:** cameras 2 and 3 and the map. Clicking a small view (or keys `1`–`4`) moves it into the large one.
7. **ARM / DISARM.** The button must be **held for 1 second** — protection against accidental presses.
8. **Driving modes:** MANUAL, HOLD, GUIDED, AUTO, RTL, SMART RTL; the rest are in the “Other modes…” list.
9. **PC radio** — turns on driving with a Radiomaster transmitter connected to the PC over USB (see section 10).
10. **STOP** — switches the rover to HOLD immediately. The `Space` key does the same.
11. **Telemetry:** mode, speed, battery, charge, heading, distance home, GPS, link quality, coordinates.
12. **Event log** — messages from the rover and the program.
13. ● **Rec** (`F8`) — trip recording: camera video and telemetry; 📷 (`F9`) — snapshot of a frame (section 7).
14. **Readiness** — overall pre-drive check status; a click opens the “Readiness to drive” panel (section 6). While recording, a red “Rec” timer appears next to it.

# 3. Connecting the rover (“Link”)

The **“Link”** section defines where the program receives MAVLink from. You can keep several links at once — for example, a radio modem and Starlink.




*The “Link” section*

1. **ArduPilot flight controller over USB.** When checked, the program finds the flight controller as soon as it is plugged in (115200 baud) and restores the connection after the flight controller reboots. Nothing to select.
2. **UDP — listen on a port.** The default link 14550: telemetry from radio-modem bridges, ESP32 boards over Wi-Fi and Mission Planner/MAVProxy arrives here. On the right — the state (“connected · N packets”) and the Connect/Disconnect/Delete buttons.
3. **TCP — client.** For SITL (127.0.0.1:5760), bridges and ESP32 in TCP mode: enter the address and port.
4. **Add link** (UDP, TCP or serial port — USB, radio modem) and **Save** — without saving, changes are lost after a restart.
5. **Custom ESP32 board.** A board running ORION firmware is detected automatically. If your firmware reports another autopilot type, enter the board SYSIDs separated by commas.
6. **Forwarding to Mission Planner / QGroundControl** (section 15). Further down the page — the “Detected vehicles” table: SYSID, type, mode, link and link quality.

## Typical connection options

| What you have | What to set up |
|---|---|
| Flight controller connected to the PC by a USB cable | Nothing: leave the “Connect automatically” box checked. If the flight controller does not appear — see the driver in the “System” section. |
| Telemetry radio modem (SiK, RFD900) in a PC USB port | “Add link” → “Serial port (USB, radio modem)” → the modem's COM port, 57600 baud. |

| | |
|---|---|
| ESP32 with ORION firmware over Wi-Fi | Connect the PC to the board's Wi-Fi (default network ORION-ROVER, password orion12345). UDP 14550 is enough. |
| Rover via Starlink / VPN / router | UDP 14550 on the PC; on the rover (in the ESP32 board settings — “ground station IP”) enter your PC's IP, because broadcast packets do not pass through a router. |
| SITL simulator | TCP 127.0.0.1 : 5760. |

# 4. The “Drive” screen

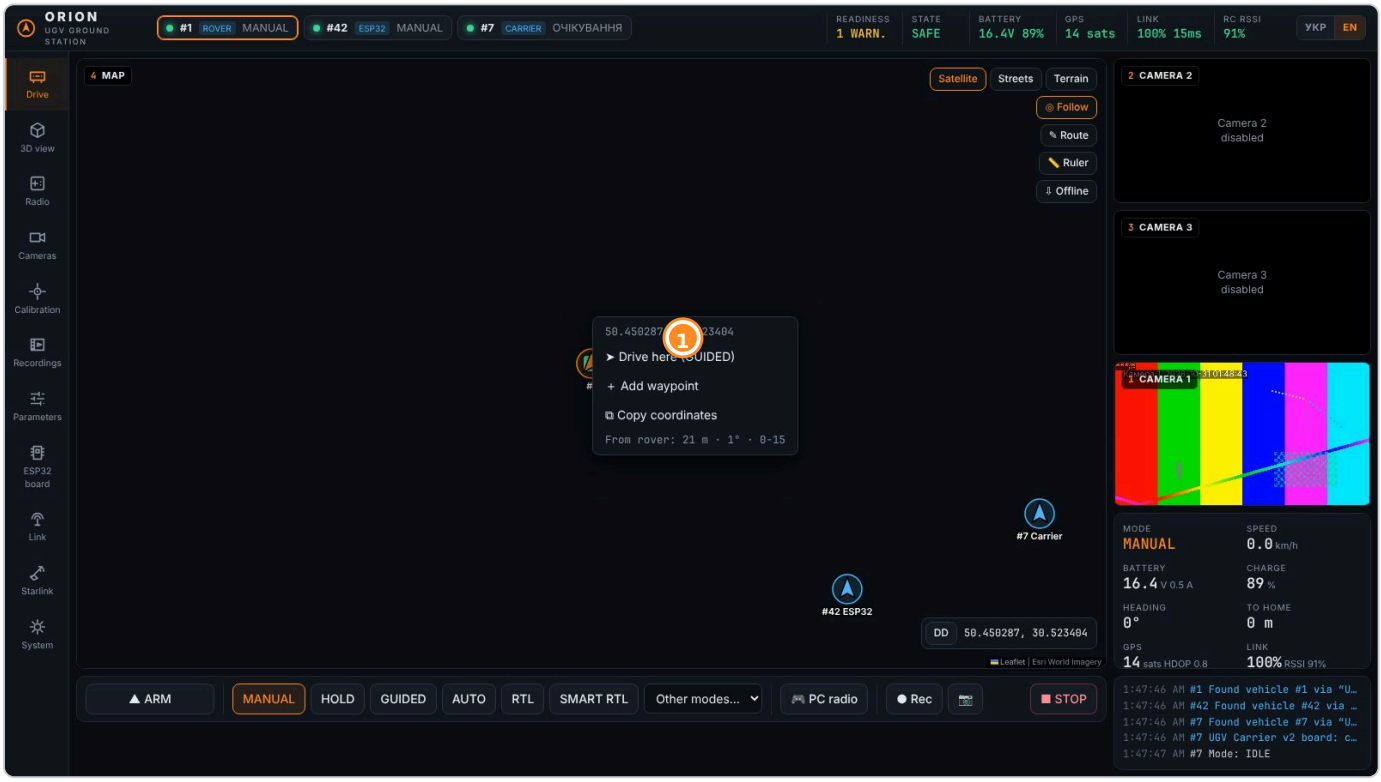
This is the main working screen (screenshot and explanation — in section 2). Order of driving:

1. Select the rover in the top bar (if there are several).
2. Check the state: GPS (navigation modes need a 3D fix), battery, link.
3. Hold **ARM** for 1 second. If the rover refuses, the reason appears in the event log (for example, *PreArm: GPS*).
4. Choose a mode: MANUAL — manual driving with the transmitter; GUIDED — drive to a point from the map; AUTO — follow a route; RTL / SMART RTL — return home.
5. To stop — **STOP** or `Space` (HOLD). When finished — hold **DISARM**.

> For ESP32 boards the MANUAL / HOLD / AUTO modes and relay buttons are available — the program shows only what the board supports.

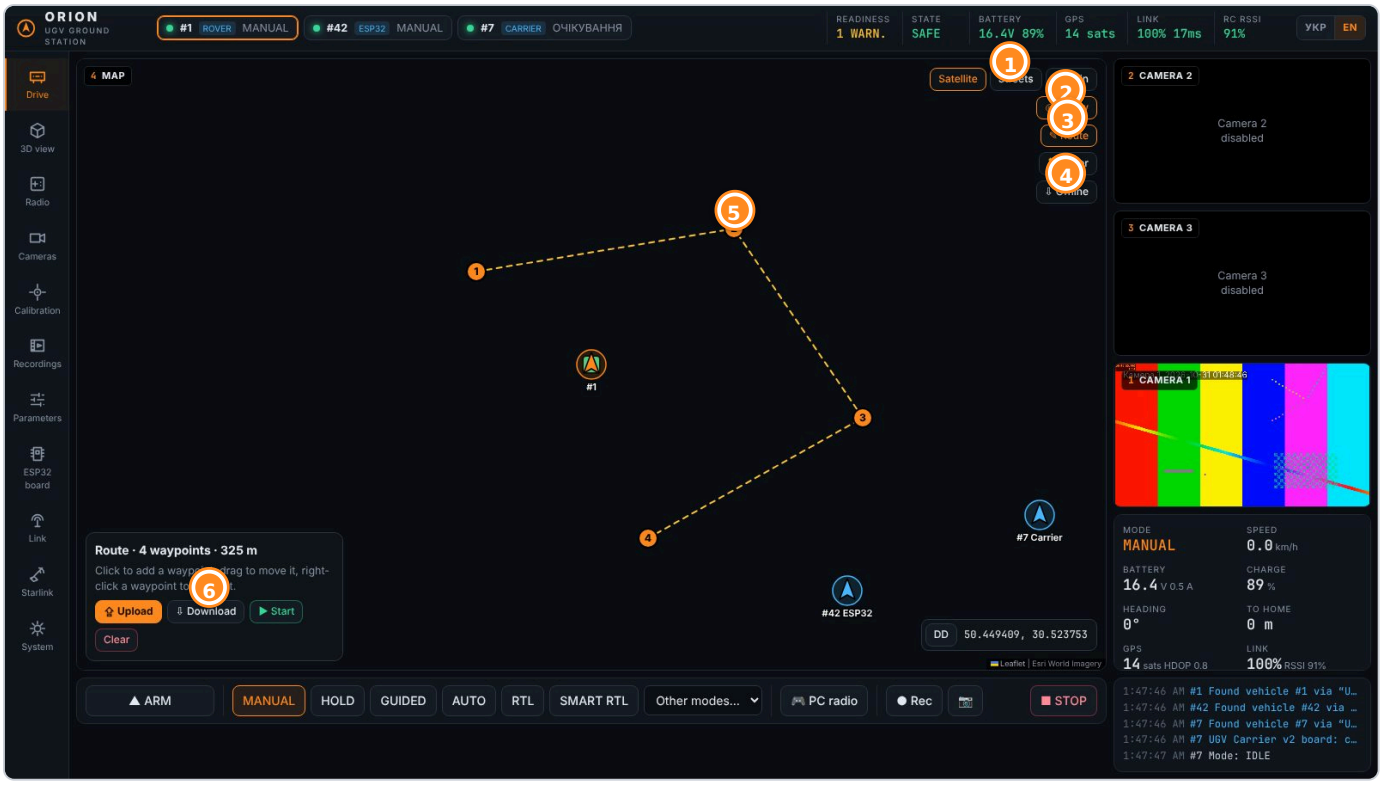
# 5. Map, coordinates, ruler and routes

To work with the map, move it into the large view (click the “MAP” tile or press `4`).



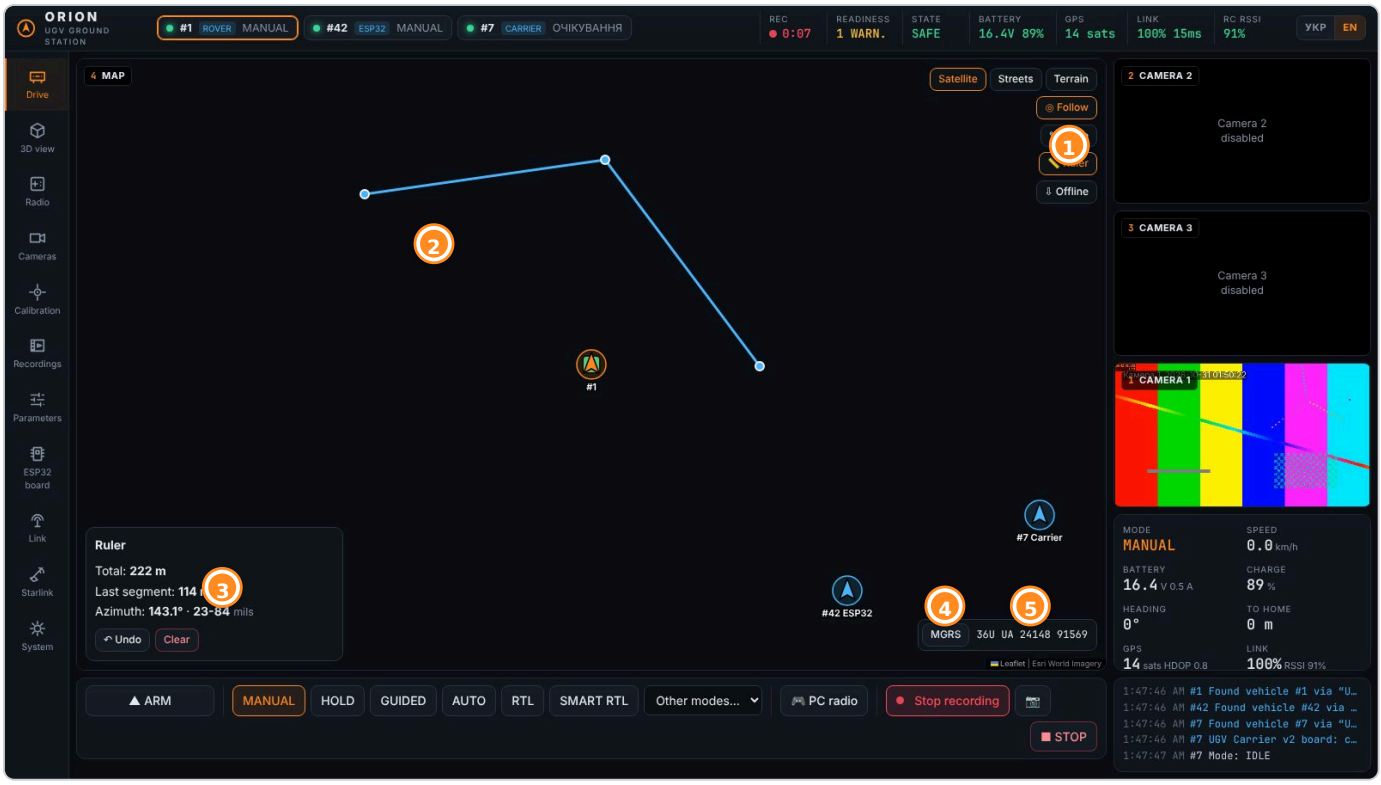

*Map context menu (right click)*

1. **Right-click on the map:** “Go here (GUIDED)” — the rover switches to GUIDED and drives to the point (must be armed); “Add route point”; “Copy coordinates”.




*“Route” mode*

1. **Map layer:** Satellite, Street, Terrain.
2. **Follow** — the map moves with the rover.
3. **Route** — turns on editing: click to add a point, drag to move, right-click a point to delete it.
4. **Offline** — save the tiles of the visible area on the PC so the map works without internet (zoom to the area you need before saving).
5. **Route points** numbered in driving order.
6. **Route panel:** number of points and length; **Upload** — write the mission to the flight controller; **Download** — read the mission from the flight controller; **Start** — start the mission (AUTO mode); **Clear**.



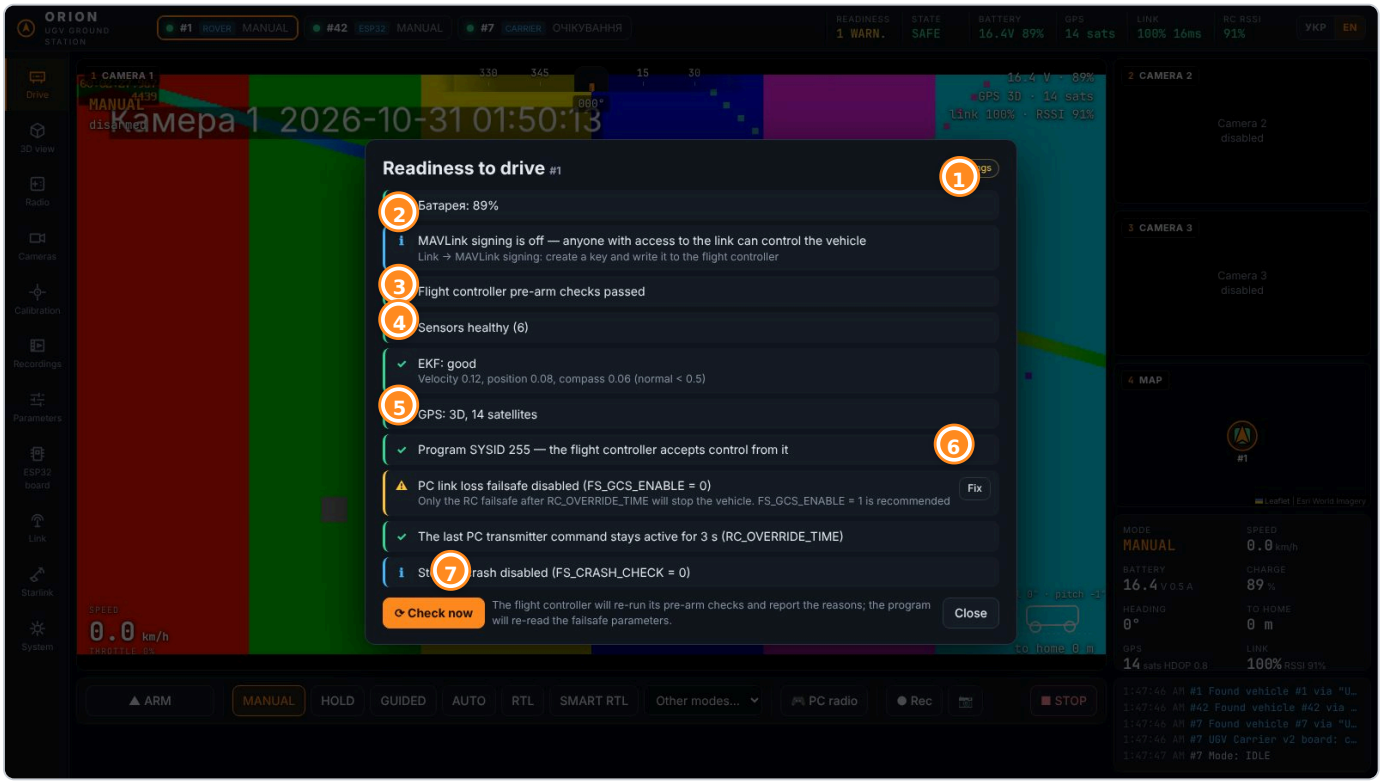

*Ruler and coordinates in MGRS format*

1. **Ruler** — turn it on and click points on the map.
2. **Ruler points**; segments are joined by a line.
3. **Ruler panel:** total length, last segment, its azimuth in degrees and in mils (6000 per circle, “26-36”). “Undo” removes the last point, “Clear” removes all.
4. **Coordinate format:** a click switches DD (degrees) → DMS (degrees, minutes, seconds) → MGRS → USK-2000 (Gauss–Krüger X/Y, as on topographic maps). The choice is remembered and applies to the map, the telemetry panel and copying.
5. **Coordinates under the cursor** (or of the rover when the cursor is off the map).

In the context menu (right click) the coordinates are shown in the chosen format; “Copy coordinates” copies exactly those. It also shows the distance and azimuth from the rover to the point. MGRS and USK-2000 conversion is checked against the same reference points as in AURA GCS (accuracy — within 1 m).

# 6. Readiness to drive

One panel instead of scattered messages: sensors, EKF, GPS, battery, flight controller pre-arm checks and failsafe parameters. The state is shown in the top bar — **Readiness: READY / N WARN. / NOT READY**; a click opens the full panel.




*The “Readiness to drive” panel*

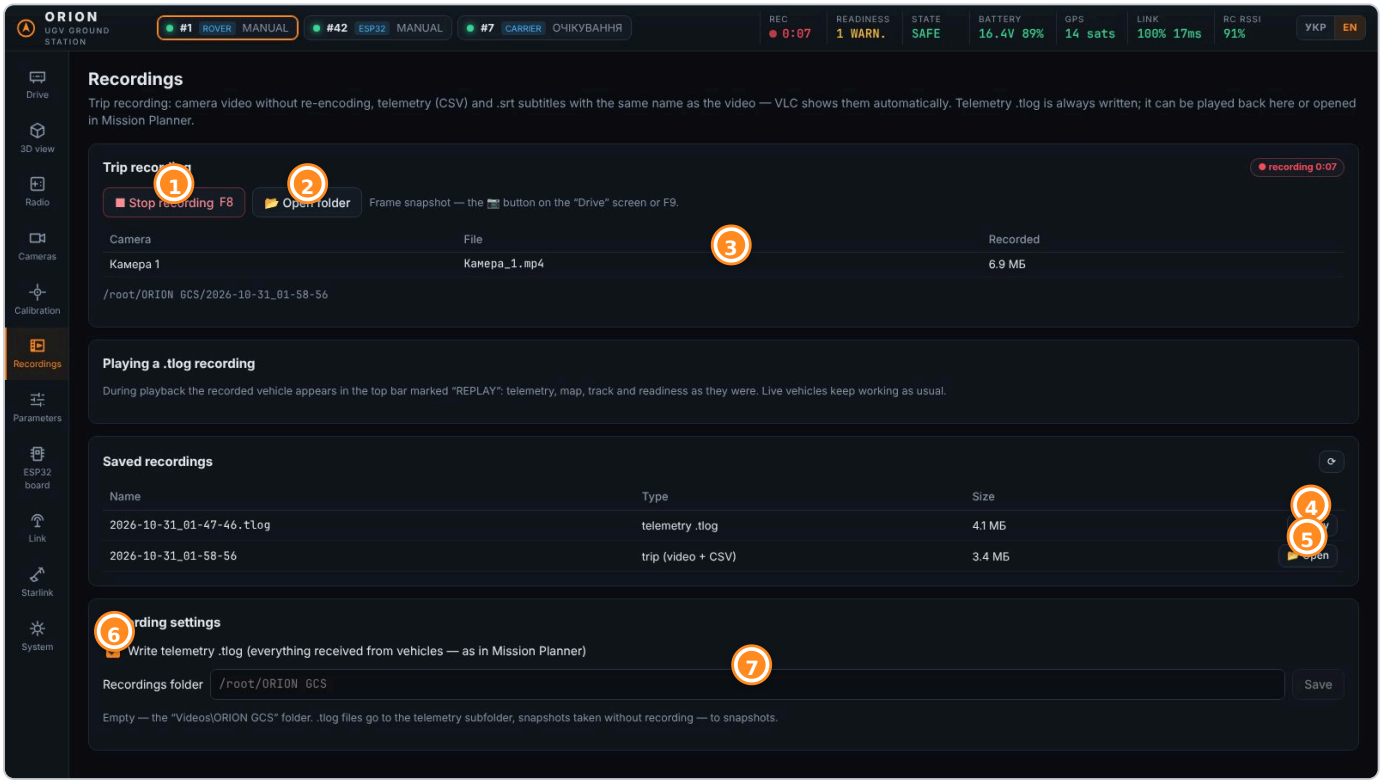
1. **Overall state:** ready / warnings / not ready.
2. **Battery** — below 40% is a warning, below 20% is “too low to go out”.
3. **Sensors** from SYS_STATUS: gyros, accelerometers, compasses, GPS, AHRS and so on. Faulty ones are listed by name.
4. **EKF** — velocity, position and compass variances (normal < 0.5) and whether an absolute position is known.
5. **Program SYSID** — the flight controller accepts PC control only from “its own” station (MAV_GCS_SYSID / SYSID_MYGCS).
6. **Fix** — for unsafe failsafe settings the program offers a recommended value and writes it to the flight controller after confirmation.
7. **Check now** — the flight controller repeats its pre-arm checks and sends the reasons (“PreArm: …”); the program re-reads the failsafe parameters.

What is checked for failsafe: the action on link loss (FS_ACTION not “nothing”), the failsafe for loss of the PC link (FS_GCS_ENABLE) or of the transmitter (FS_THR_ENABLE) is enabled, how long the last PC transmitter command stays in effect (RC_OVERRIDE_TIME, 1 s recommended), pre-arm checks disabled (ARMING_CHECK = 0), the action on EKF failure and stopping on collision.

# 7. Trip recording, snapshots and .tlog

Trip recording — the ● **Rec** button on the “Drive” screen (or `F8`). The session folder “Videos\ORION GCS\yyyy-mm-dd_hh-mm-ss” receives:

- **video of every enabled camera** — without transcoding, the same stream as on screen (MP4, survives a crash of the program);
- **telemetry.csv** — once a second: time, coordinates and MGRS, heading, speed, mode, ARM, battery, GPS, link quality, events (mode change, ARM/DISARM, snapshots);
- **.srt subtitles** with the same name as the video: VLC and MPC overlay time, mode, coordinates, speed and battery on the video by themselves;
- **snapshots** — the 📷 button or `F9` (a frame of the camera in the large view). Without a recording, snapshots go to the snapshots subfolder.




*The “Recordings” section*

1. **Start / Stop recording** (`F8`). Top right — the recording time.
2. **Open folder** of the current session (or of all recordings) in Explorer.
3. **Cameras in the recording:** file and how much is recorded. RTSP cameras via go2rtc and WebRTC (WHEP) cameras are written to a .ts file without transcoding: go2rtc on this PC connects to the camera as one more viewer. If the link to the camera drops, recording resumes as soon as it is back; the pause in the file is a frozen frame.
4. ▶ **Play** the .tlog telemetry.
5. **Open** the folder of a saved trip.
6. **Write .tlog telemetry** — always on by default: everything received from the vehicles, in Mission Planner format (telemetry subfolder).
7. **Recordings folder** — another folder (for example, on a separate drive).

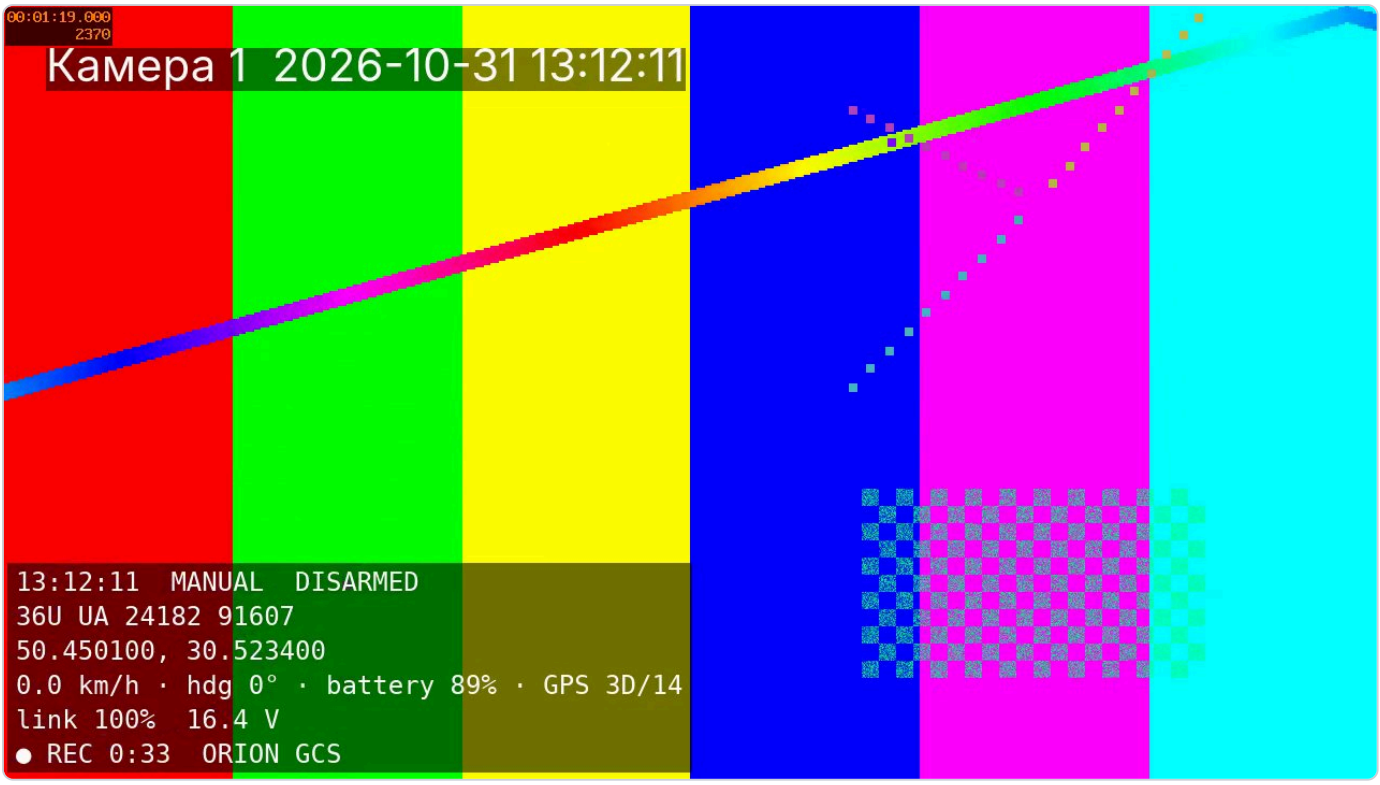



*.tlog playback*

1. **The vehicle from the recording** appears in the top bar marked “REPLAY” (▶1). Telemetry, map, trail, HUD and readiness show what happened during the trip. Live vehicles keep working alongside as usual.
2. **File name** of the recording.
3. **Pause / play.**
4. **Seek** — drag the slider.
5. **Speed:** ×0.5 … ×10. Long pauses in the recording are compressed to 2 s.
6. **Close** — the vehicle from the recording disappears.

## Video with telemetry in the frame

The .srt subtitles are visible only in a player that supports them. If the video goes into a report, a messenger or YouTube, enable **“Also video with telemetry burned into the frame”** in “Recordings” → “Recording settings”. A `<camera>_osd.mp4` file then appears next to the clean recording, with the telemetry burned into the picture.




*A frame from “Камера_1_osd.mp4” (camera name from the settings)*

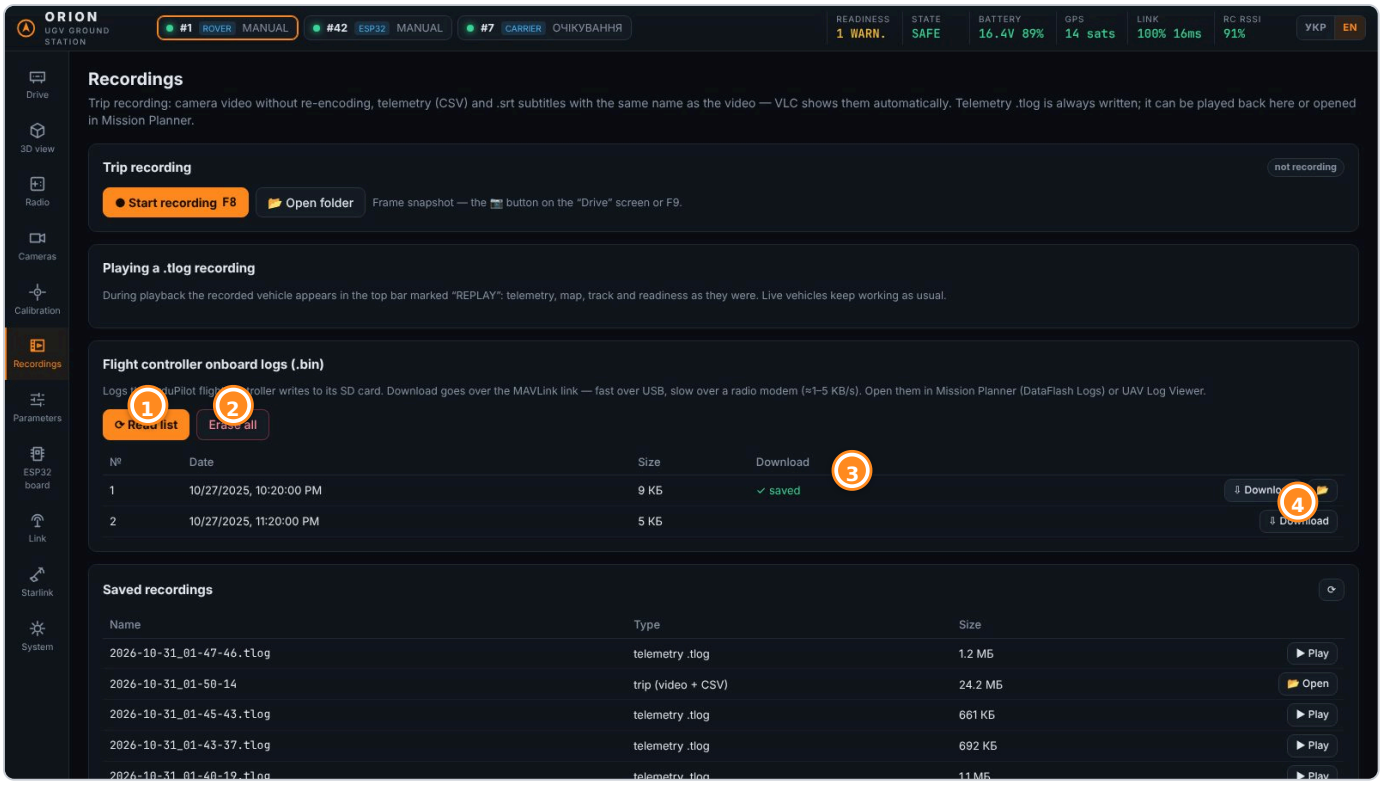
- **Bottom left of the frame:** time, mode, ARMED/DISARMED (and “NO LINK” if the link is lost), MGRS and decimal coordinates, speed, heading, battery, GPS, link quality and voltage, recording time. The text updates every second.
- **Text language** — the interface language on the device where the checkbox was turned on (Ukrainian or English).
- **The clean recording** without transcoding is written as before — the original is not lost.

Burning in means transcoding (H.264): about one CPU core per 1080p camera. On a weak PC enable it only for the camera you need (turn the others off while recording) or use the .srt subtitles.

During playback no commands are sent to the vehicle from the recording — the mode and ARM buttons only show a message for it. .tlog files also open in Mission Planner (Telemetry Logs → Tlog).

## Flight controller onboard logs (.bin)

The ArduPilot flight controller writes detailed logs to its SD card. You can fetch them directly from ORION, without Mission Planner.



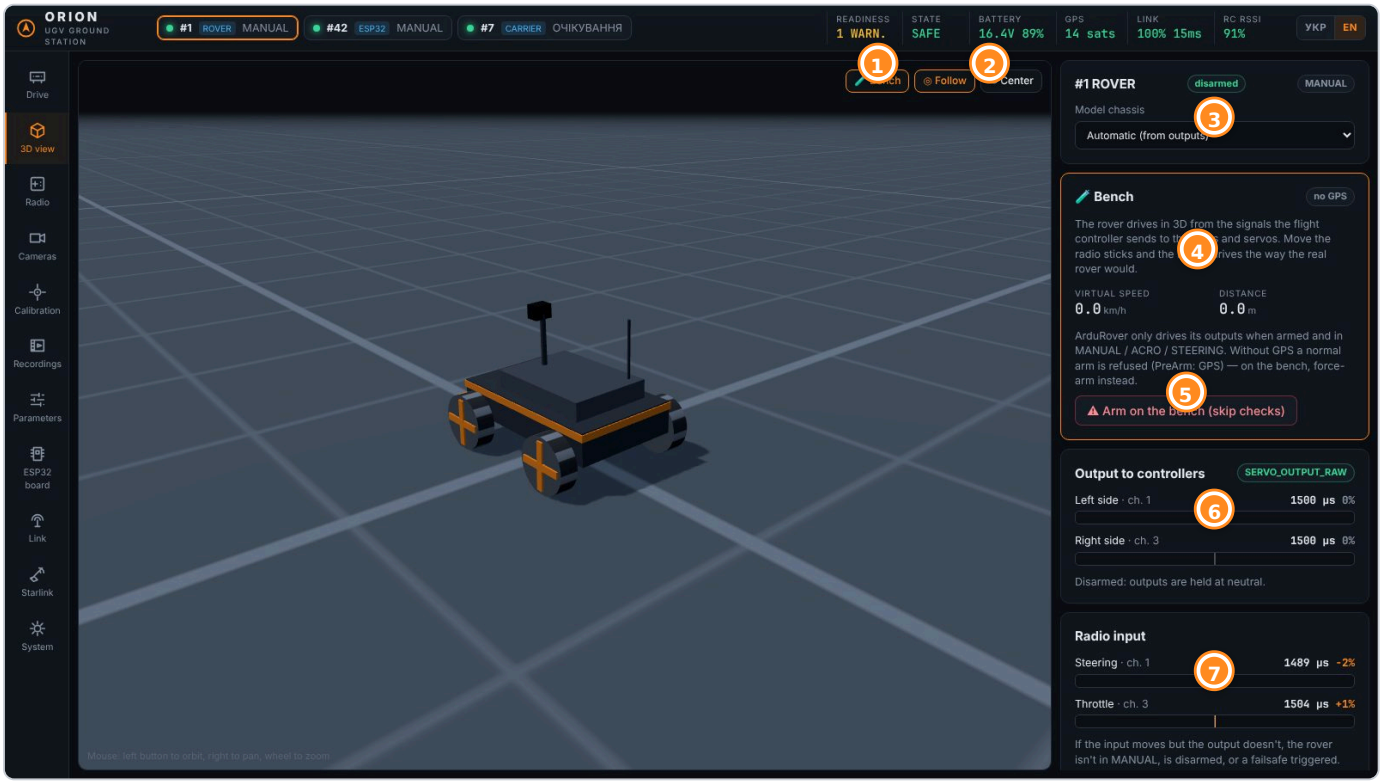

*"Recordings" section → "Flight controller onboard logs (.bin)"*

1. **Read list** of logs on the SD card of the active vehicle.
2. **Erase all** — free the card (with confirmation).
3. **Download state:** percent, "✓ saved" or an error. Missing chunks are re-requested automatically. The 📂 button opens the `logs` folder inside the recordings folder.
4. **Download** the log (or "Cancel" while downloading).

> Over USB a log of tens of megabytes downloads in a minute or two; over a radio modem only ≈1–5 KB/s, so large logs are better fetched by cable. .bin files open in Mission Planner (DataFlash Logs) or on the UAV Log Viewer website.

# 8. “3D view” and bench check

The section shows a 3D model of the rover that moves by real telemetry, as well as what the flight controller sends to the ESCs and servos and what comes from the transmitter. Handy for checking the setup without going out.



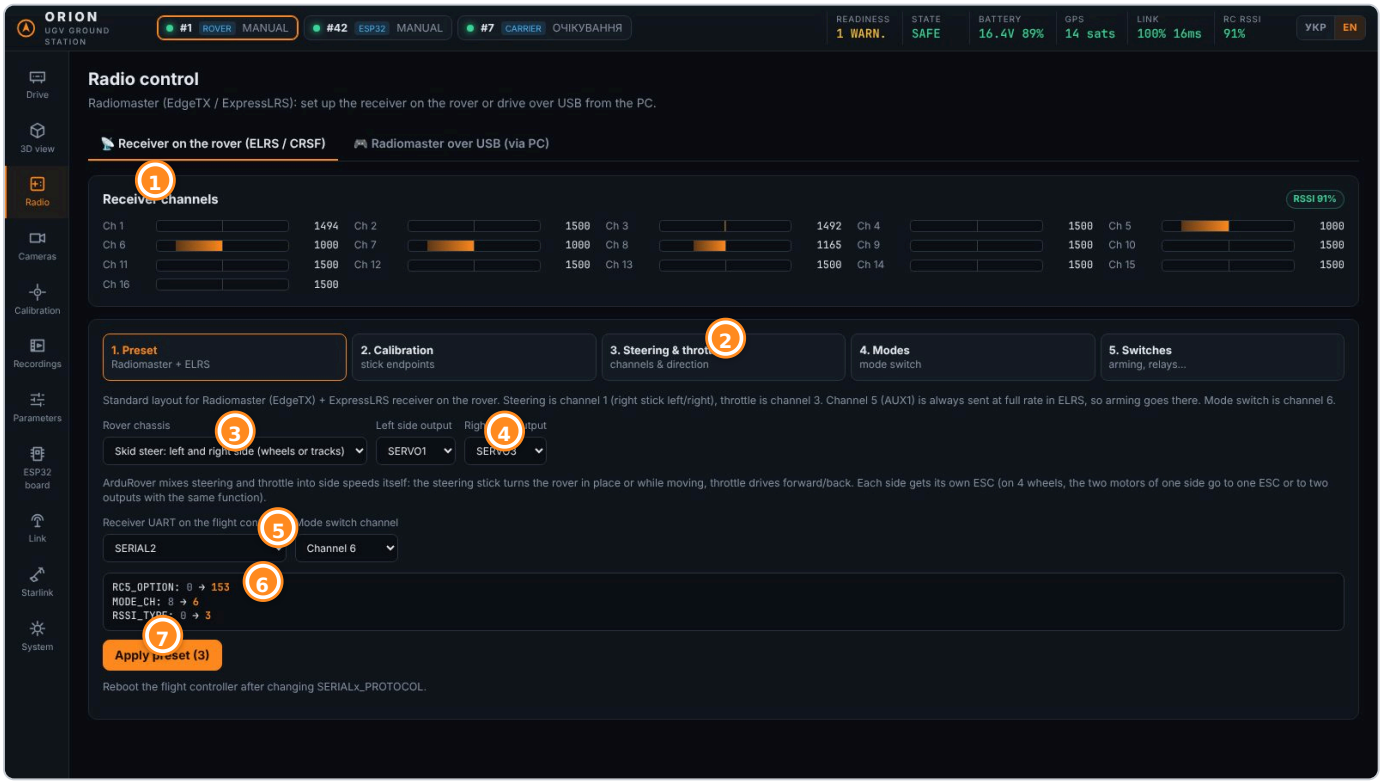

*“3D view” with the bench mode on*

1. **Bench** — the tabletop check mode: the model drives by the signals the flight controller sends to its outputs. Move the sticks — the model drives the way the real rover would.
2. **Follow / Center** — the camera follows the model or the view is reset. Mouse: left button — rotate, right — pan, wheel — zoom.
3. **Model chassis:** automatic (from the output functions) or manual — skid steering or car steering.
4. **Bench panel:** virtual speed and distance travelled.
5. **Arm on the bench (skip checks)** — ArduRover drives its outputs only when armed, and arming is refused without GPS. On the bench you can force-arm. **The wheels must be off the ground!**
6. **Output to controllers** (SERVO_OUTPUT_RAW): PWM to the left/right side (or steering/throttle) in µs and percent.
7. **Radio input:** steering and throttle from the receiver. If the input moves but the output does not, the rover is not in MANUAL, is disarmed or a protection has triggered.

The bench check also works with a flight controller connected to the PC only by a USB cable, without GPS: this way you can set up the transmitter and see the response before mounting on the chassis.

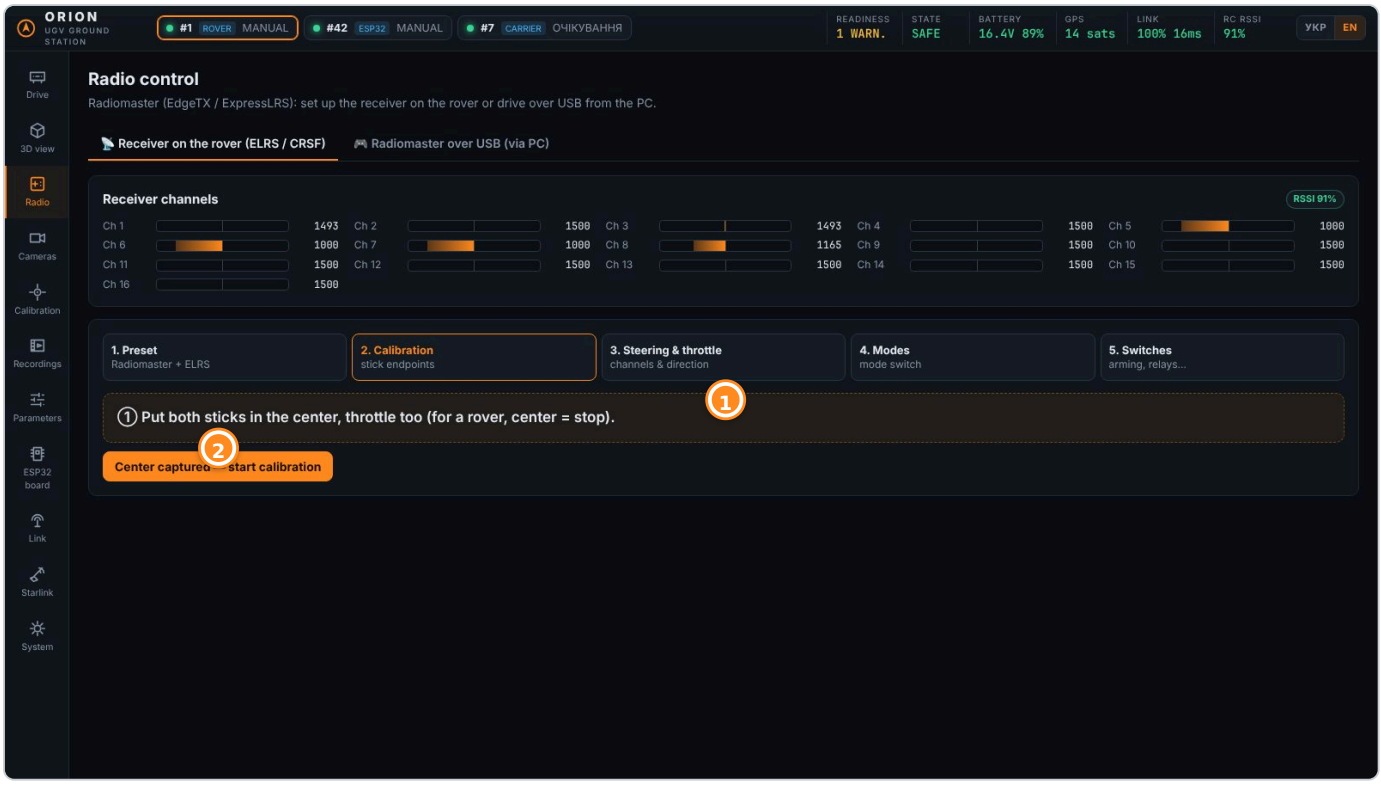
# 9. Radio: receiver on the rover (ELRS / CRSF)

The **“Receiver on the rover (ELRS / CRSF)”** tab sets up a Radiomaster transmitter whose receiver is on the rover and connected to the flight controller. The wizard has five steps. First the program reads the parameters from the flight controller (5–20 seconds).



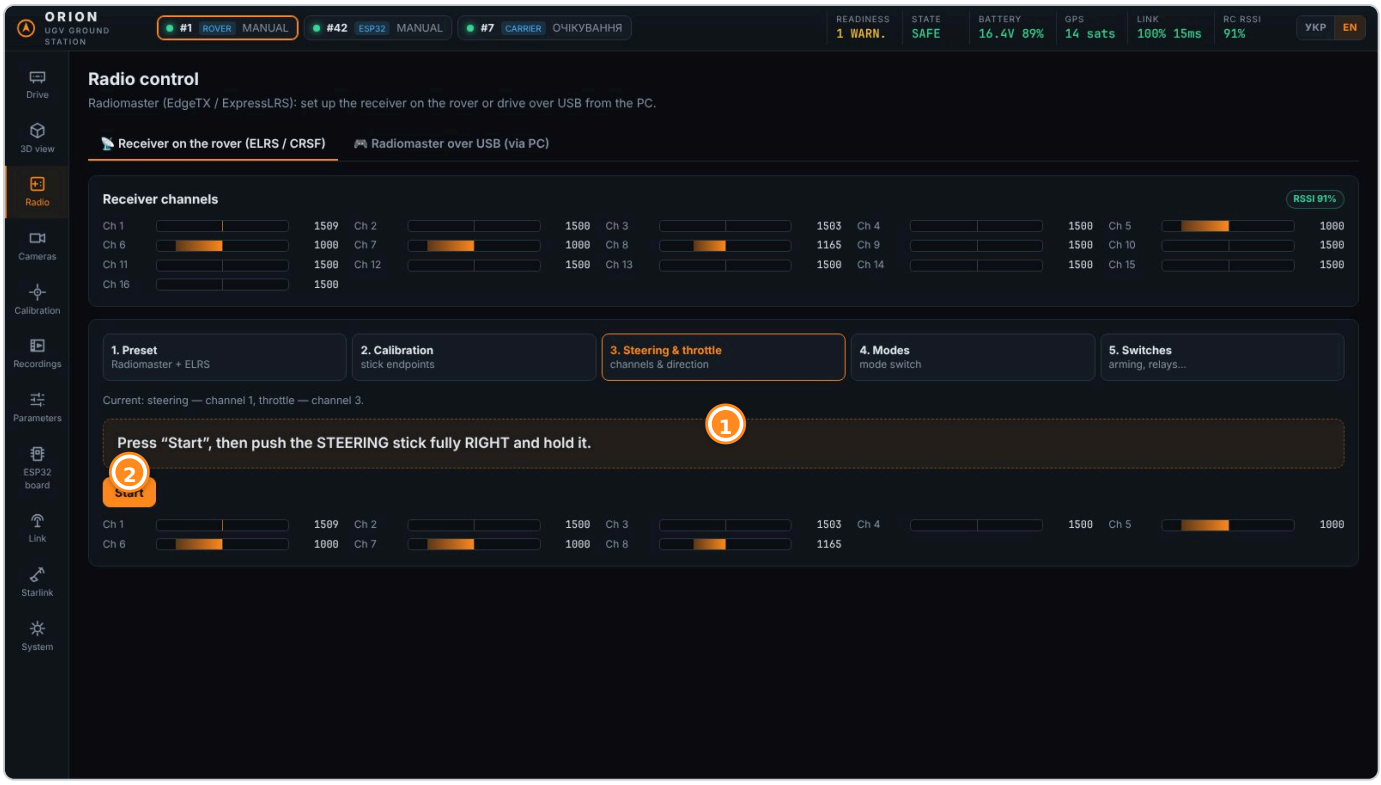

*Step 1. Preset*

1. **Receiver channels** — live values of 16 channels. Move a stick — its bar should move.
2. **Wizard steps.** Go through them in order or open any one.
3. **Rover chassis.** “Skid steering: left and right side (wheels or tracks)” — for most UGVs without a steering axle; “Car steering” — steering servo + throttle.
4. **Flight controller outputs:** for skid steering — the left and right side outputs (SERVOx_FUNCTION = 73 / 74), for car steering — steering and throttle (26 / 70). On 4 wheels the two motors of one side connect to one ESC or to two outputs with the same function.
5. **Receiver UART** on the flight controller (the SERIALx where ELRS is connected) and the **mode switch channel**.
6. **Preview:** which parameters will change (was → will be).
7. **Apply preset.** After changing SERIALx_PROTOCOL the flight controller must be rebooted (button in the “Parameters” section).



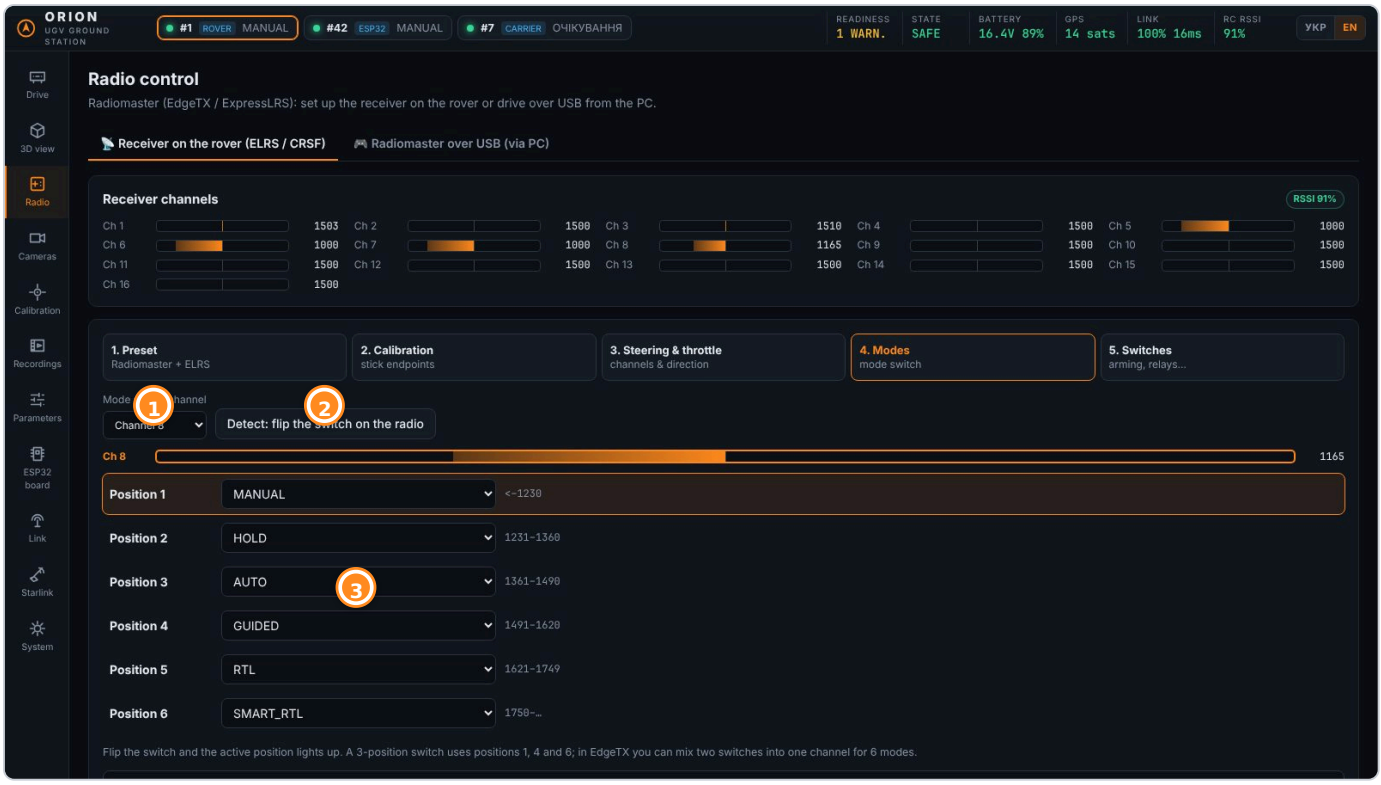

*Step 2. Stick calibration*

1. Instructions for the current step. Put both sticks in the center (throttle too: for a rover, center = stop).
2. Press **“Center fixed — start calibration”**, move the sticks to the end stops in all directions and press “Done”. The program writes RCx_MIN / MAX / TRIM.



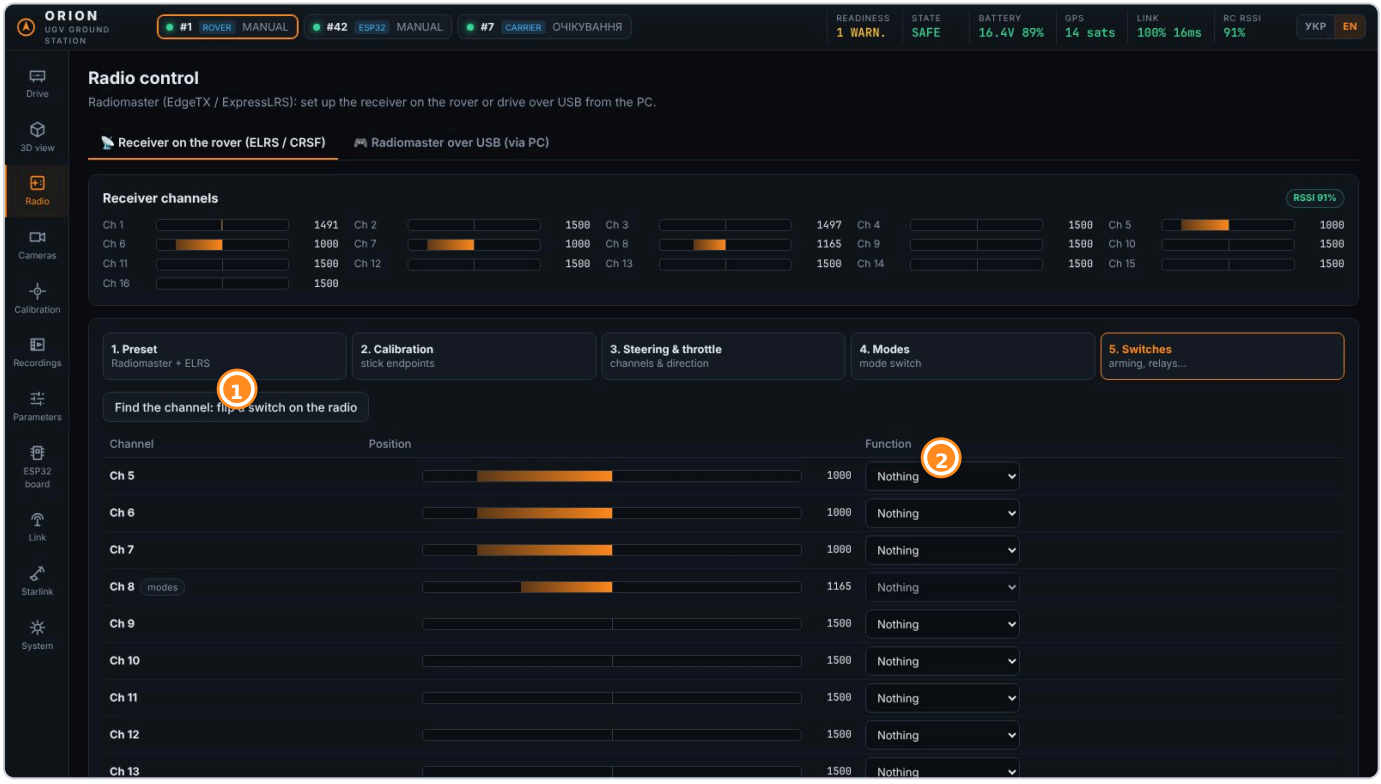

*Step 3. Steering and throttle*

1. The program tells you what to do: press “Start” and turn the **steering stick right** to the stop, then push **throttle forward**.
2. From the stick movement the program determines which channel is steering and which is throttle, and whether reversing is needed.




*Step 4. Modes*

1. **Mode switch channel** (MODE_CH).
2. **Detect** — flick the switch on the transmitter and the program finds the channel itself.
3. **Six positions** and a mode for each. The active position is highlighted. A 3-position switch uses positions 1, 4 and 6.




*Step 5. Switches*

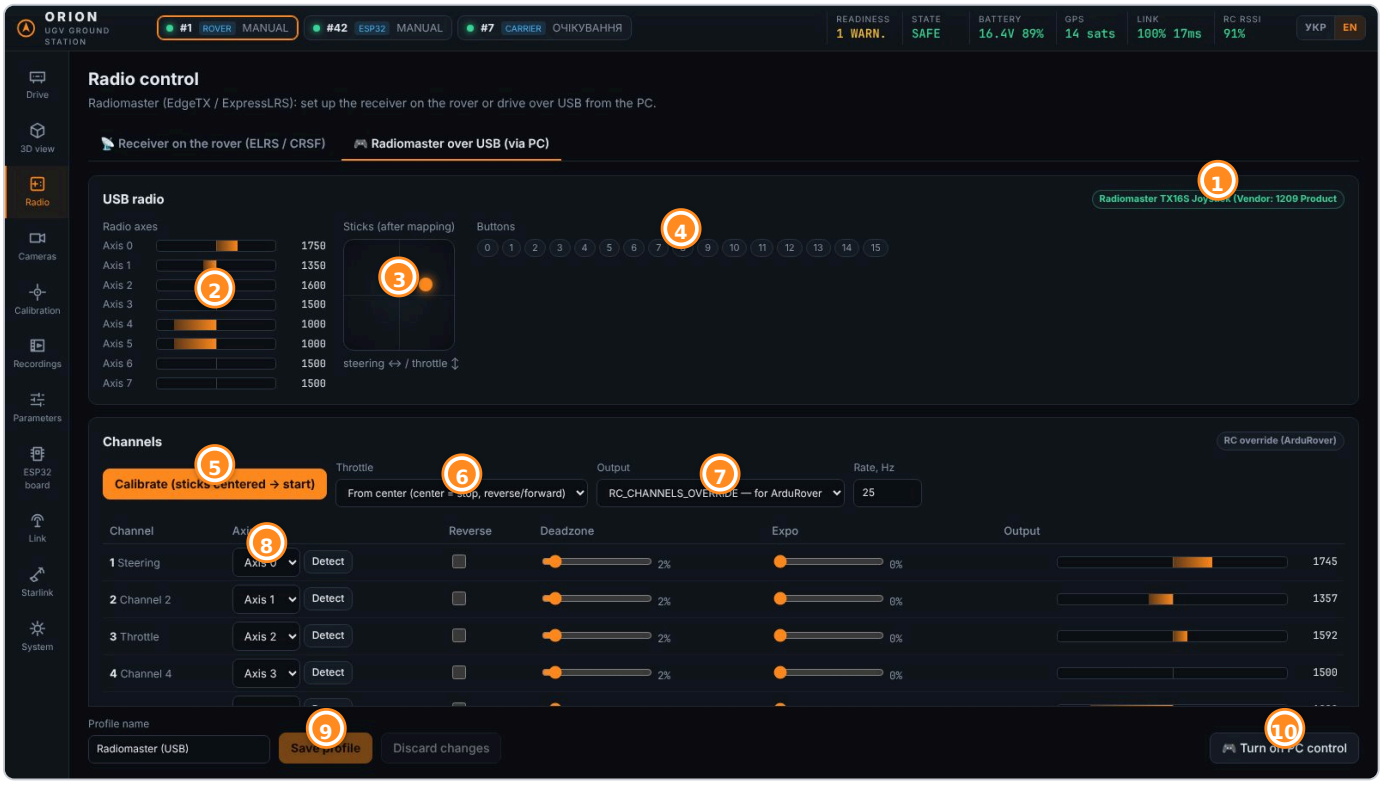
1. **Find channel** — flick the switch you need and its channel row lights up.
2. **Channel function** (RCx_OPTION): arm/disarm, relay, save waypoint, RTL and so on.

# 10. Radio over USB: driving from the PC

The **“Radiomaster over USB (via PC)”** tab lets you drive the rover with a transmitter connected to the PC by cable: the program reads the transmitter as a joystick and sends commands to the rover over the telemetry link (RC override).

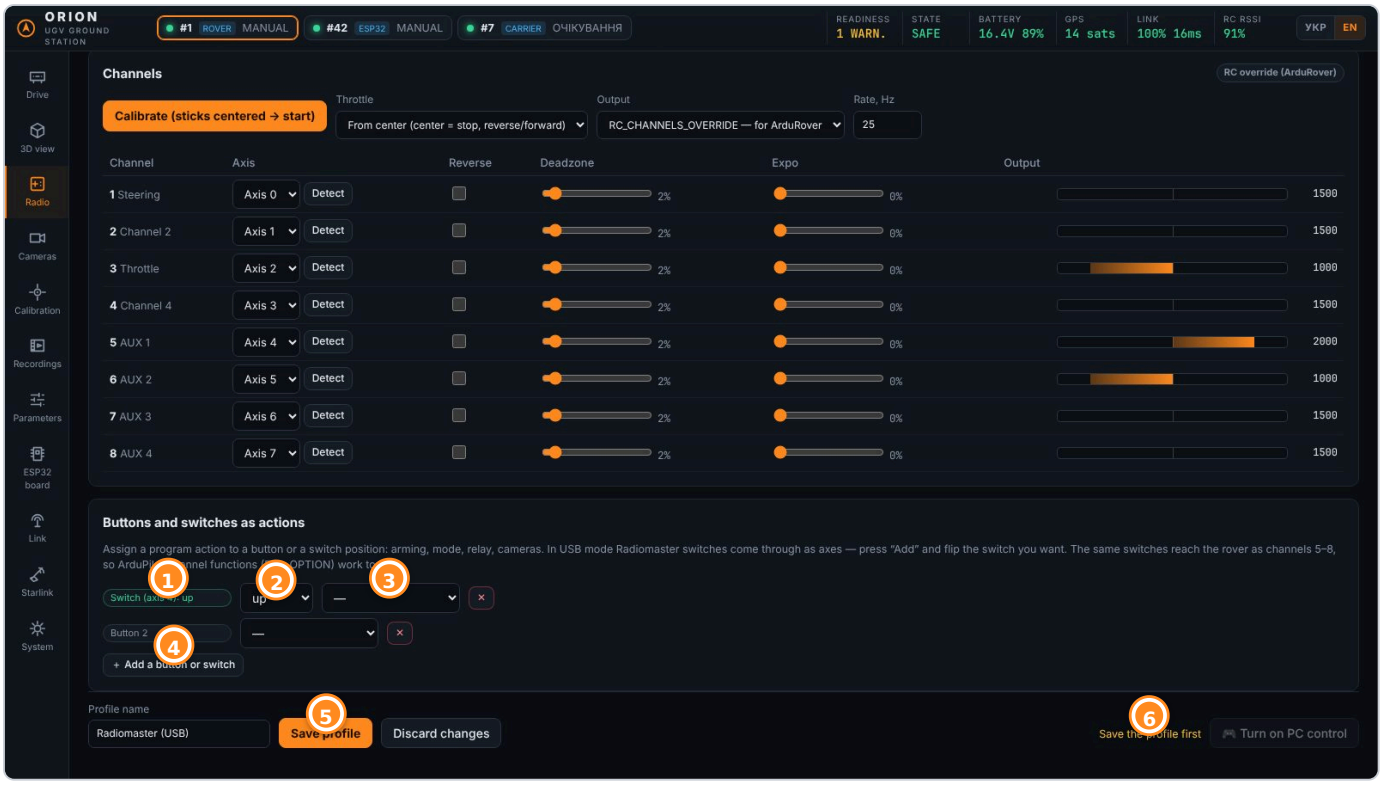
## Preparing the transmitter

1. Connect the Radiomaster to the PC with a USB-C cable. On the transmitter screen choose **USB Joystick (HID)**.
2. In EdgeTX: SYS → USB → **Classic** mode or **Advanced with 8 axes**.
3. Move a stick or press a button — the system shows the transmitter only after the first action.




*Radio over USB: axes and channels*

1. **Device** — name of the detected transmitter (green mark = connected).
2. **Transmitter axes** — raw values of all axes.
3. **Sticks after setup** — where steering and throttle will go.
4. **Buttons** — light up when pressed.
5. **Calibrate** — sticks to center, press, move them to the end stops.
6. **Throttle:** “From center” (center = stop, back/forward) or “From bottom” (bottom = stop, forward only).
7. **Output:** RC_CHANNELS_OVERRIDE — for ArduRover; send rate (25 Hz by default).
8. **Channel table:** which axis goes to which channel (the “Detect” button — move the stick you need), reverse, deadzone, expo and the output value.
9. **Save profile** — required after changes.
10. **Turn on PC control** — active only after the profile is saved.




*Buttons and switches as actions*

1. **Binding:** “Switch (axis 4): up” or “Button 2”.
2. **Switch position:** up / middle / down. In USB mode Radiomaster switches are sent as axes, so each position can have its own action.
3. **Program action:** arm, disarm, HOLD (stop), RTL (return home), mode…, relay (toggle), next camera, swap cameras.
4. **Add a button or switch** — press it, then press the button or flip the switch on the transmitter: the program detects it itself.
5. **Save profile.**
6. A hint on why PC control is not available yet (for example, “Save the profile first”).

> The same switches go to the rover as channels 5–8, so ArduPilot channel functions (RCx_OPTION) work too. Enabled PC control is visible on the “Drive” screen — the “PC radio: ON” button and a label in the HUD.

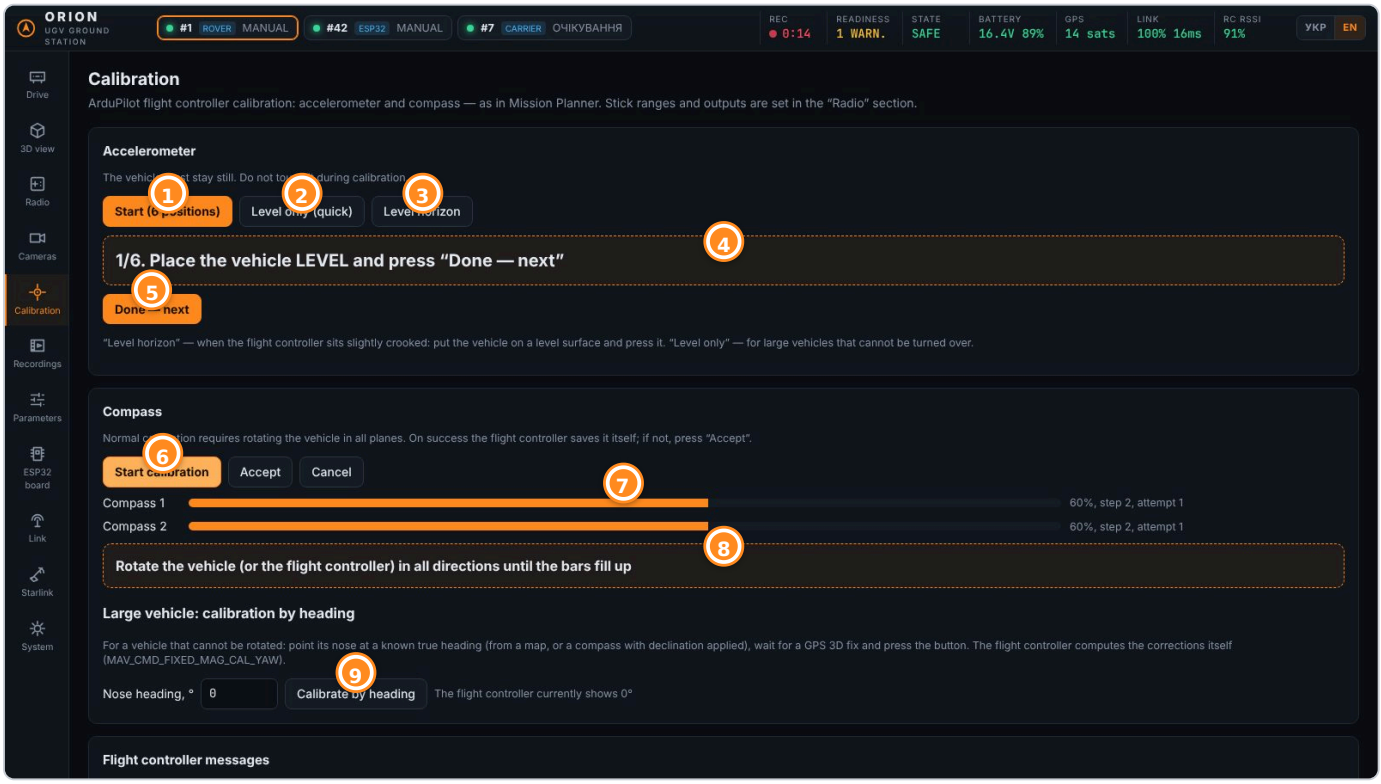
## Speed limit and cruise

When PC control is on, two items appear on the “Drive” screen panel:

- ⚡ **100 / 75 / 50 / 30 %** — speed limit: the share of throttle travel sent to the rover. Full stick forward at 50% gives half throttle — handy for driving in a column or for training.
- **Cruise (C)** — hold the current throttle so you do not have to hold the stick: push the throttle forward and press the button, the `C` key or a transmitter button assigned to “Cruise”. Steering works as usual. Cruise turns off with the same button, by moving the throttle stick back, with the `Space` key or by turning PC control off.

# 11. Accelerometer and compass calibration

The **“Calibration”** section — ArduPilot flight controller calibration as in Mission Planner. The vehicle must be connected and disarmed. Transmitter stick limits and outputs are in the “Radio” section (section 9).

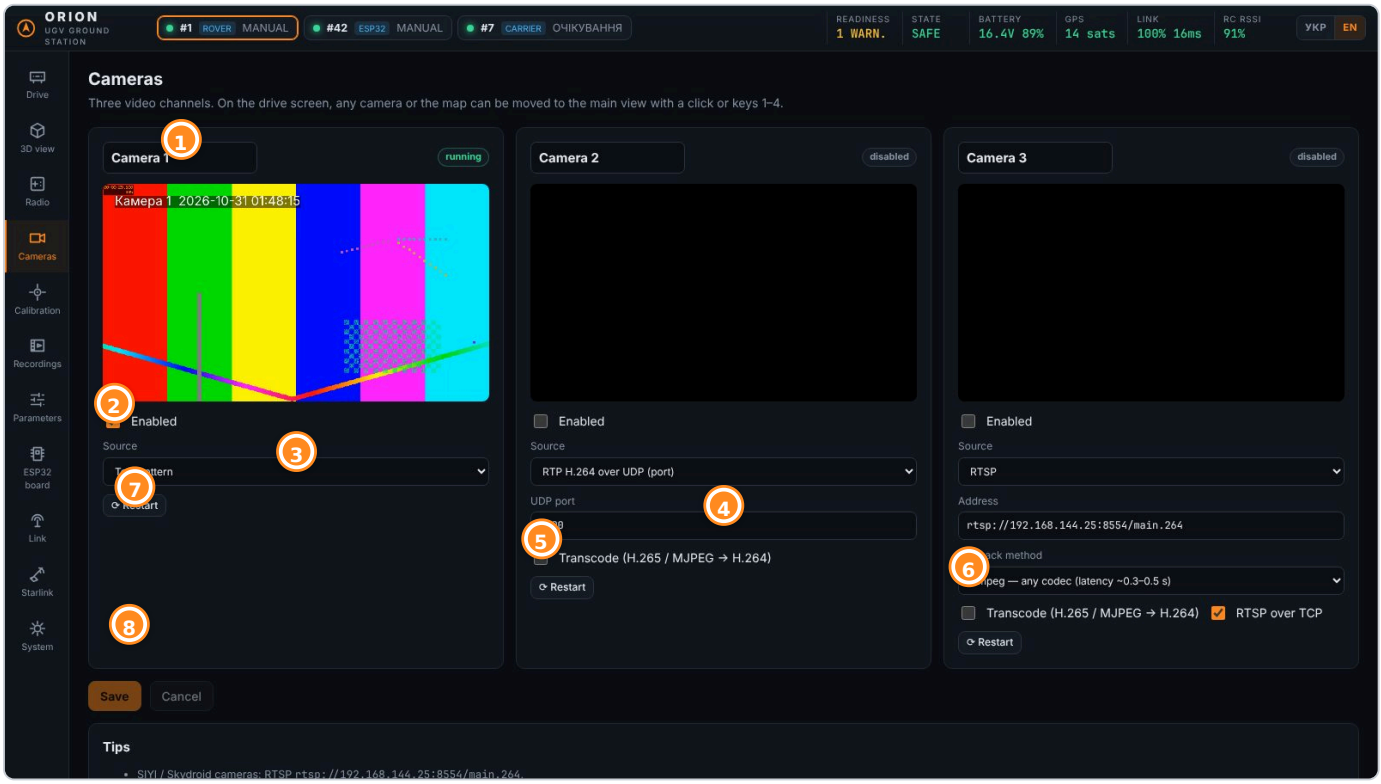



*The “Calibration” section*

1. **Start (6 positions)** — full accelerometer calibration.
2. **Level only (quick)** — for large vehicles that cannot be turned over: it is enough for the vehicle to stand level.
3. **Level horizon** — when the flight controller is mounted slightly crooked: put the vehicle on a level surface and press.
4. **Flight controller prompt:** which position to put the vehicle in (level, left side, right side, nose down, nose up, upside down).
5. **Done — next** — press when the vehicle is in the required position. At the end — “Accelerometer calibration successful. Reboot the flight controller”.
6. **Compass:** “Start calibration” and rotate the vehicle in all planes; “Accept” — if the flight controller did not save the result itself; “Cancel”.
7. **Progress of each compass**, at the end — quality (fitness, lower is better) and whether it was saved.
8. **Hint** on what to do now.
9. **Large vehicle: calibration by heading.** Point the vehicle at a known true heading (by map or compass corrected for declination), wait for a GPS 3D fix, enter the heading and press — the flight controller computes the offsets itself.

After calibration reboot the flight controller (“Parameters” section → “Reboot rover”). Further down the page — flight controller messages during calibration.

# 12. Cameras

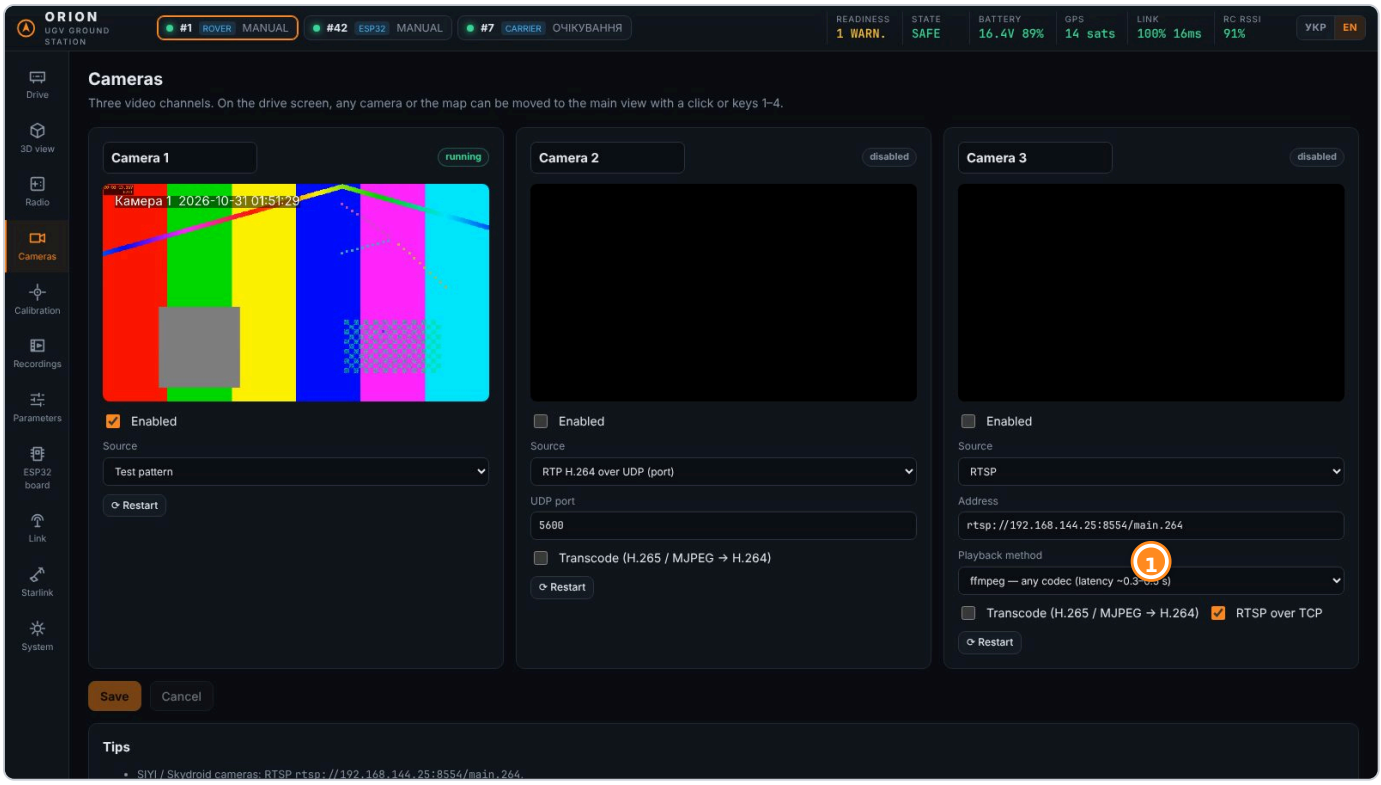



*The “Cameras” section*

1. **Camera name** — how it is labelled on the drive screen.
2. **Enabled.**
3. **Source:** RTSP (IP cameras, SIYI, Skydroid), RTP H.264 or MPEG-TS over UDP (Raspberry Pi/companion, as in QGroundControl), SRT, WebRTC/WHEP (mediamtx, go2rtc — lowest latency), MJPEG over HTTP (ESP32-CAM), USB camera or capture card, test signal.
4. **Address or UDP port** — e.g. `rtsp://192.168.144.25:8554/main.264` or port 5600.
5. **Transcode** H.265 / MJPEG → H.264 — if the camera sends a format that does not display. Loads the CPU.
6. **RTSP over TCP** — more stable on a poor link.
7. **Restart** the camera stream.
8. **Save** the settings of all cameras.

Further down the page there are hints with addresses for common cameras. Video needs ffmpeg — it is already in the installer (see section 20).

## WebRTC via go2rtc — lowest latency



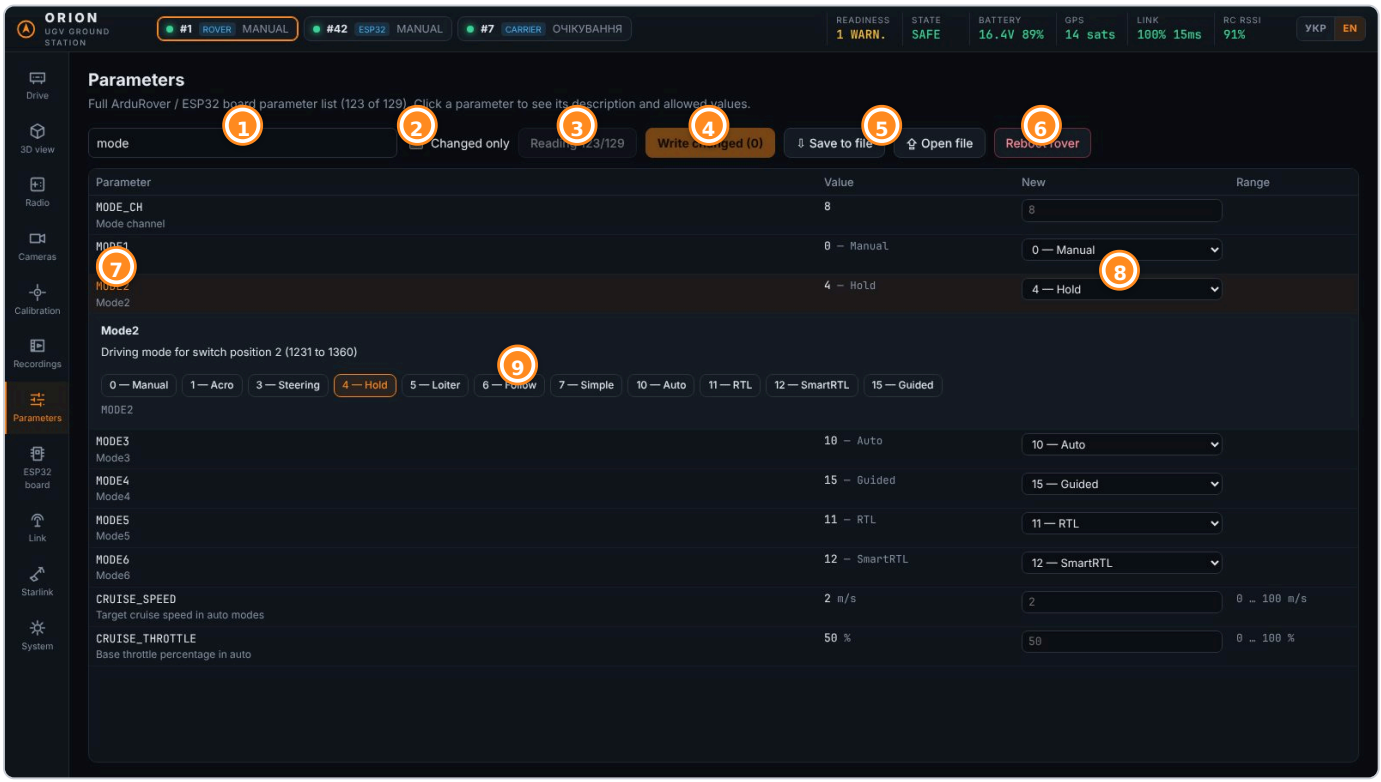

*RTSP camera: playback method*

1 **Playback method:** “ffmpeg” (any codec, latency 0.3–0.5 s) or “WebRTC via go2rtc on this PC” (latency ≈0.1–0.2 s).

go2rtc is a small program bundled with the installer; ORION starts it by itself when at least one RTSP camera uses “WebRTC via go2rtc”. WebRTC in the program window shows only H.264: if the camera sends H.265, set H.264 in the camera or enable “Transcode”. Trip recording works for such cameras too (a .ts file without transcoding). The go2rtc state is shown in the “System” section.

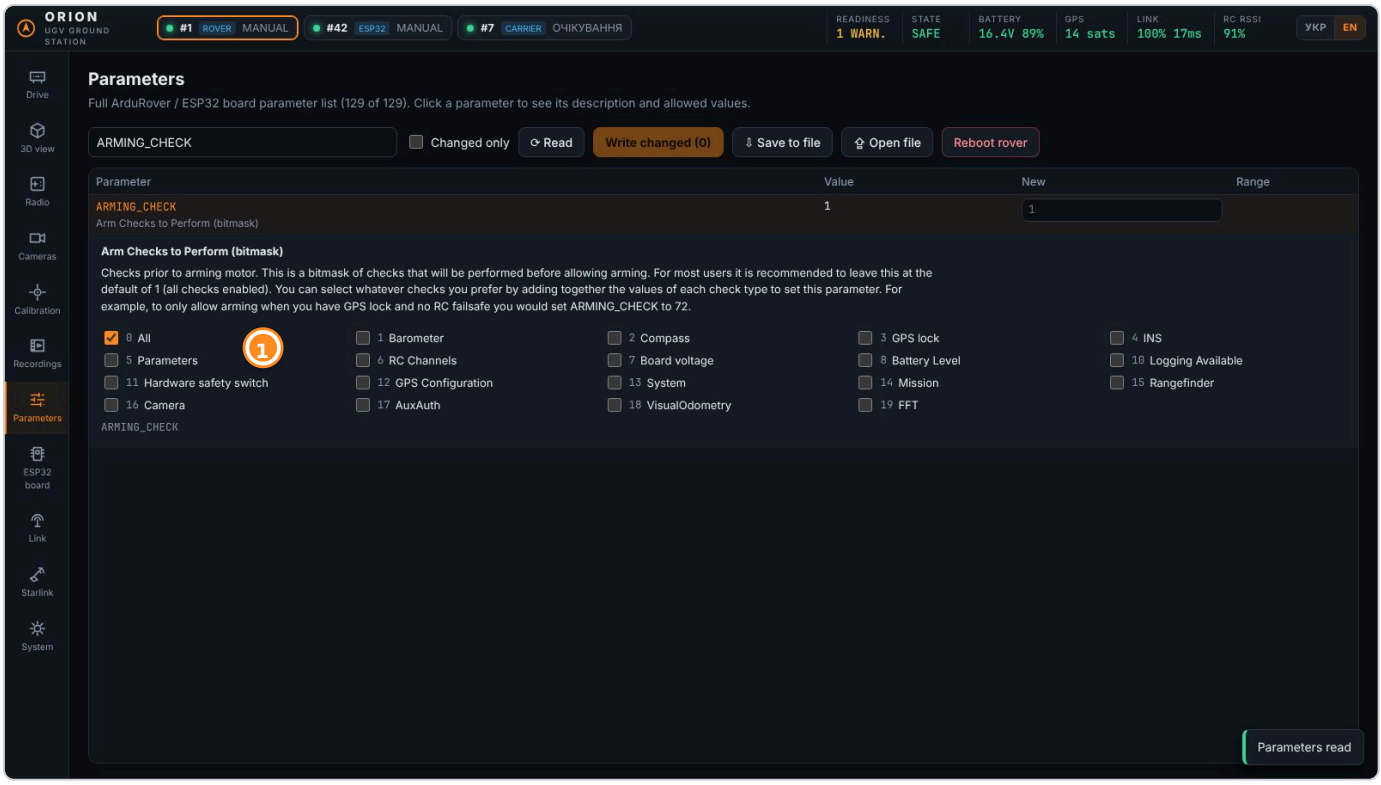
# 13. Parameters

The full parameter list of an ArduRover flight controller or an ESP32 board. Every parameter has a caption and a description in the interface language (Ukrainian or English), units and the allowed range.



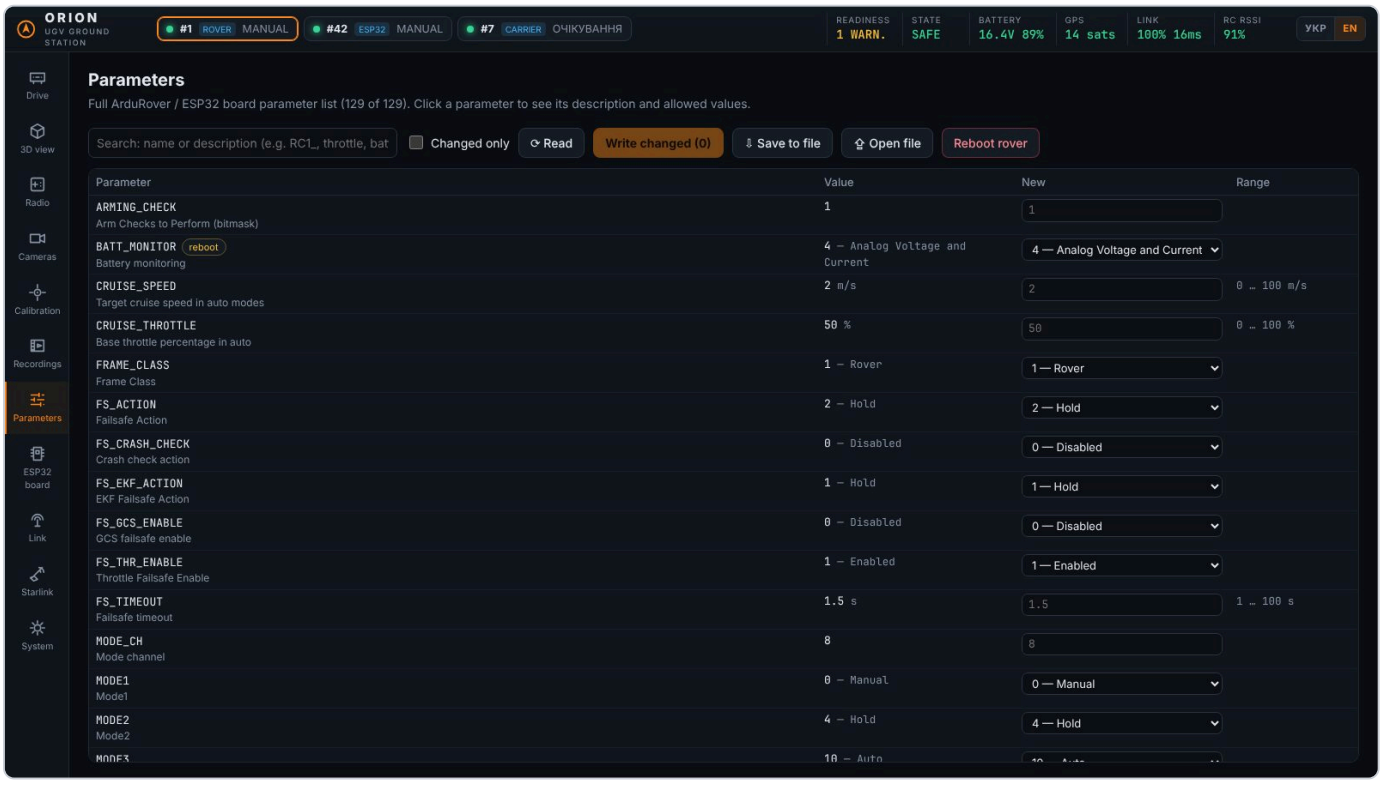

*The “Parameters” section: search and value list*

1. **Search** by name (MODE, RC1_), caption or description in Ukrainian or English (“battery”, “throttle”, “mode”).
2. **Changed only** — show only the parameters you changed but have not written yet.
3. **Read** — load parameters from the rover (shows progress).
4. **Write changed** — send all changes to the rover with one click (the count is in brackets).
5. **Save to file / Open file** — a backup (.param, compatible with Mission Planner) and loading from a file. Values from a file are first shown as changes — review them and write.
6. **Reboot rover** — needed after parameters marked “reboot”.
7. **Parameter name and caption.** A click expands the description.
8. **New value:** a drop-down list for enumerations, a field for numbers (out-of-range values turn red). `Enter` writes a single parameter at once.
9. **Expanded description** with buttons for all allowed values.




*Bitmask: each bit is a separate checkbox*

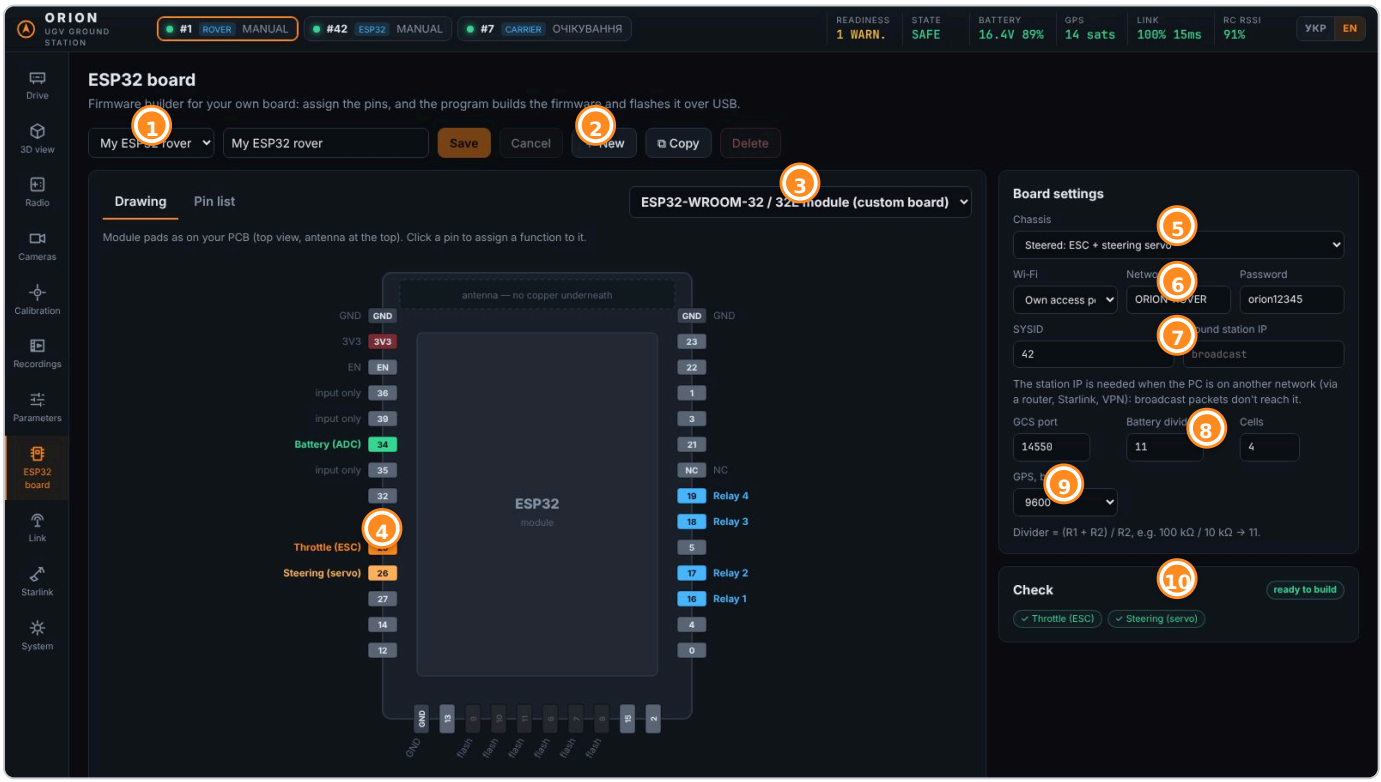
1 For bitmasks (ARMING_CHECK, LOG_BITMASK and so on) every bit is shown as a labelled checkbox — no need to add up the sum by hand.




*The same section in English (EN switch)*

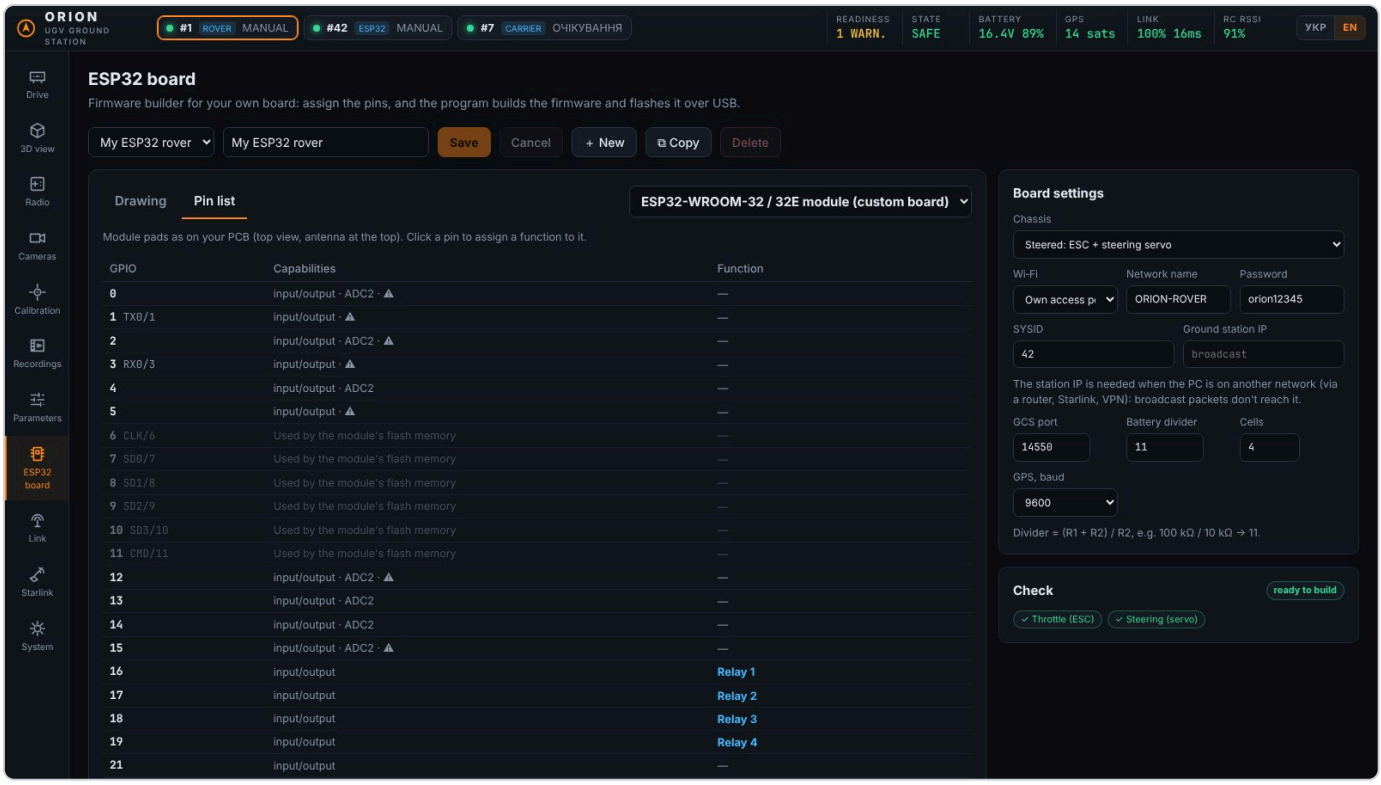
# 14. ESP32 board: firmware builder

A firmware builder for your own ESP32 board: assign the pins and the program builds and flashes the firmware over USB. You can keep several board profiles.

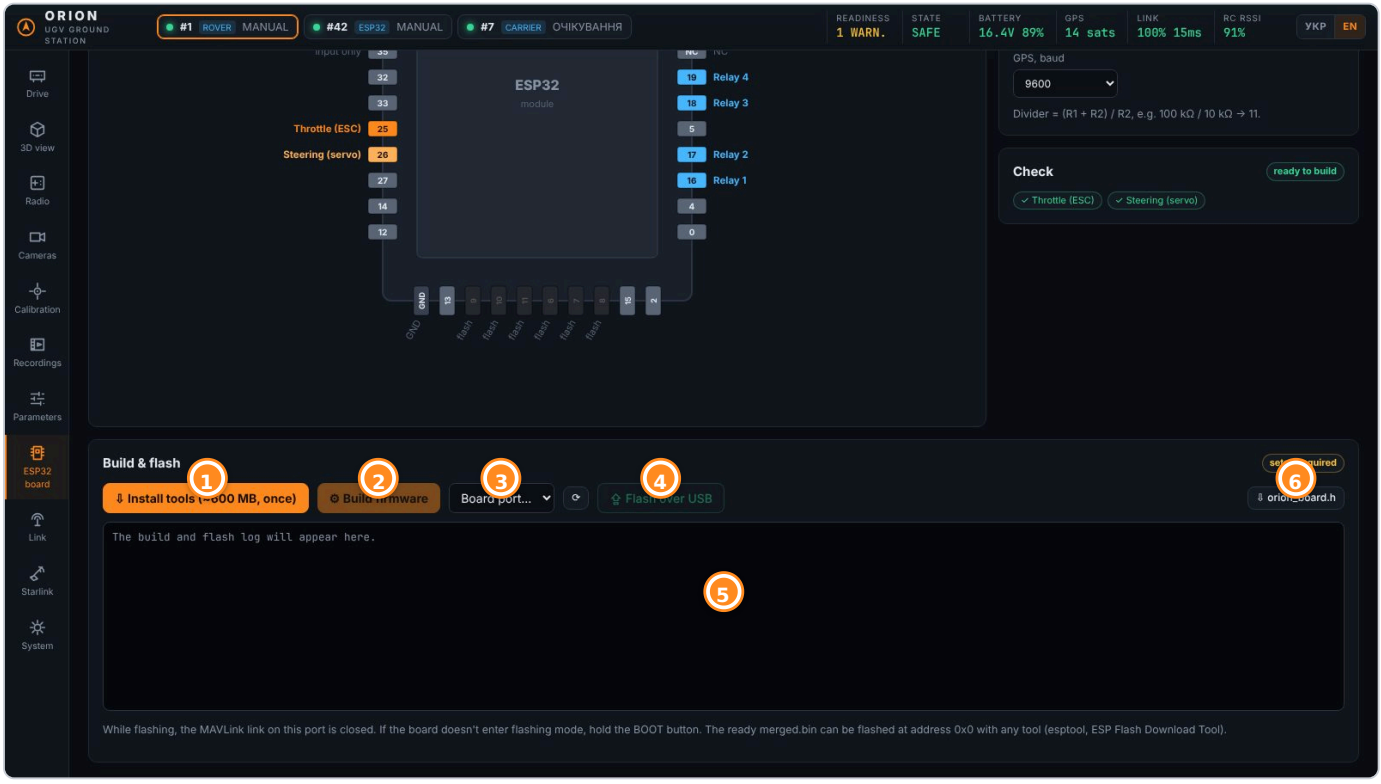



*The “ESP32 board” section*

1. **Board profile** and its name.
2. **Save / Cancel / New / Copy / Delete.**
3. **Module:** ESP32-WROOM-32/32E, ESP32 DevKitC, ESP32-S3-WROOM-1, WT32-ETH01 (with Ethernet) — this determines the drawing and the available pins.
4. **Module drawing** (top view, antenna at the top). Click a pin to assign a function: throttle (ESC), steering (servo), left/right side, relay, battery (ADC), GPS and so on. Grey pins are taken by flash memory; “input only” pins are not suitable for outputs.
5. **Chassis:** skid steering (two ESCs, PWM + direction, H-bridge) or car steering (ESC + steering servo).
6. **Wi-Fi:** the board is an access point (its own network) or a client of your network; network name and password.
7. **SYSID** of the board (each vehicle has its own) and the **ground station IP** — needed if the PC is on another network (router, Starlink, VPN).
8. **GCS port, battery divider and cell count** — to compute voltage and charge. Divider = (R1 + R2) / R2.
9. **GPS speed.**
10. **Check:** whether all required functions are assigned and there are no pin conflicts (“ready to build”).




*“Pin list” tab: capabilities of every GPIO and the assigned function*

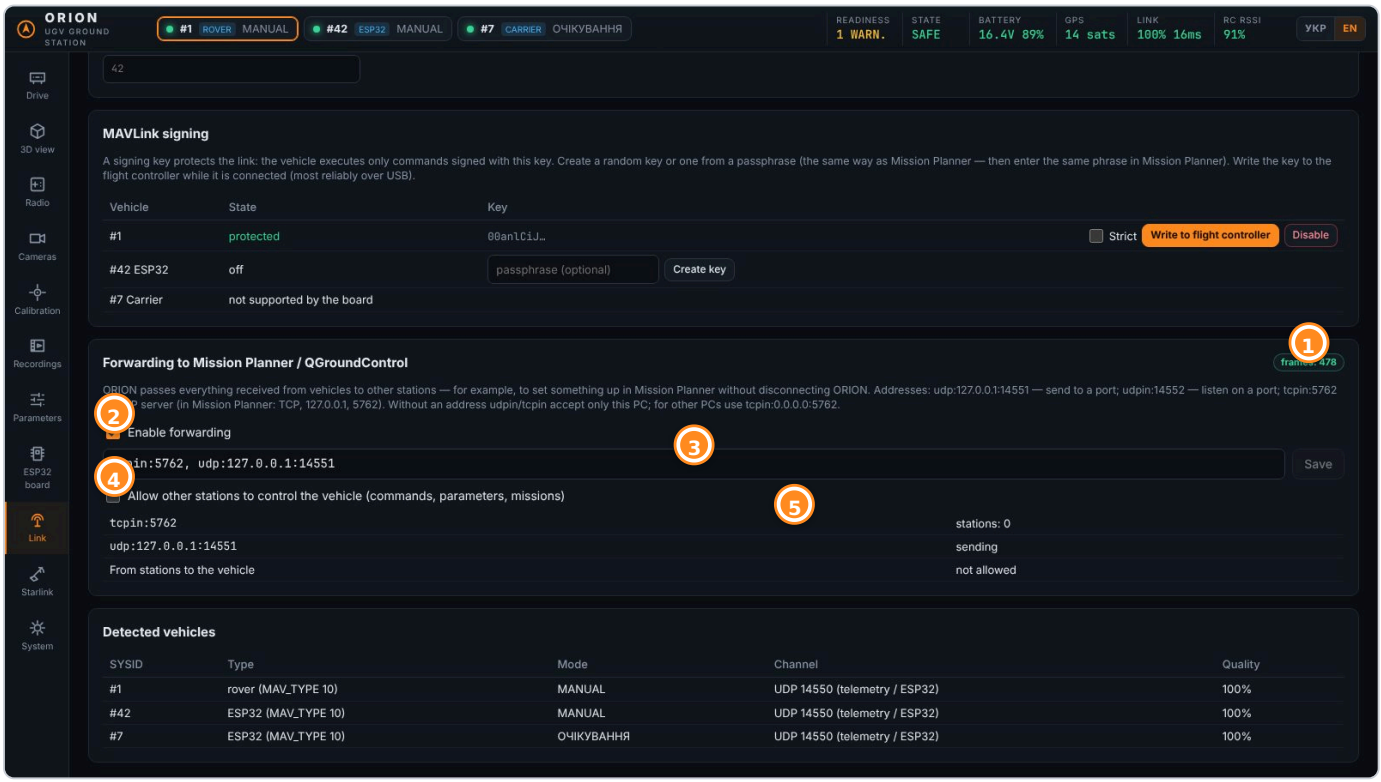



*Build and flash*

1. **Install tools** (≈600 MB, once, internet required).
2. **Build firmware.**
3. **Board port** (COM) and the button to refresh the port list.
4. **Flash over USB.** If the board does not enter flashing mode, hold its BOOT button.
5. **Log** of compiling and flashing.
6. **orion_board.h** — download the generated configuration file (for building manually in the Arduino IDE).

# 15. Forwarding to Mission Planner / QGroundControl

ORION can pass everything received from the vehicles on to other stations — for example, to set something up in Mission Planner without closing ORION or re-plugging cables.

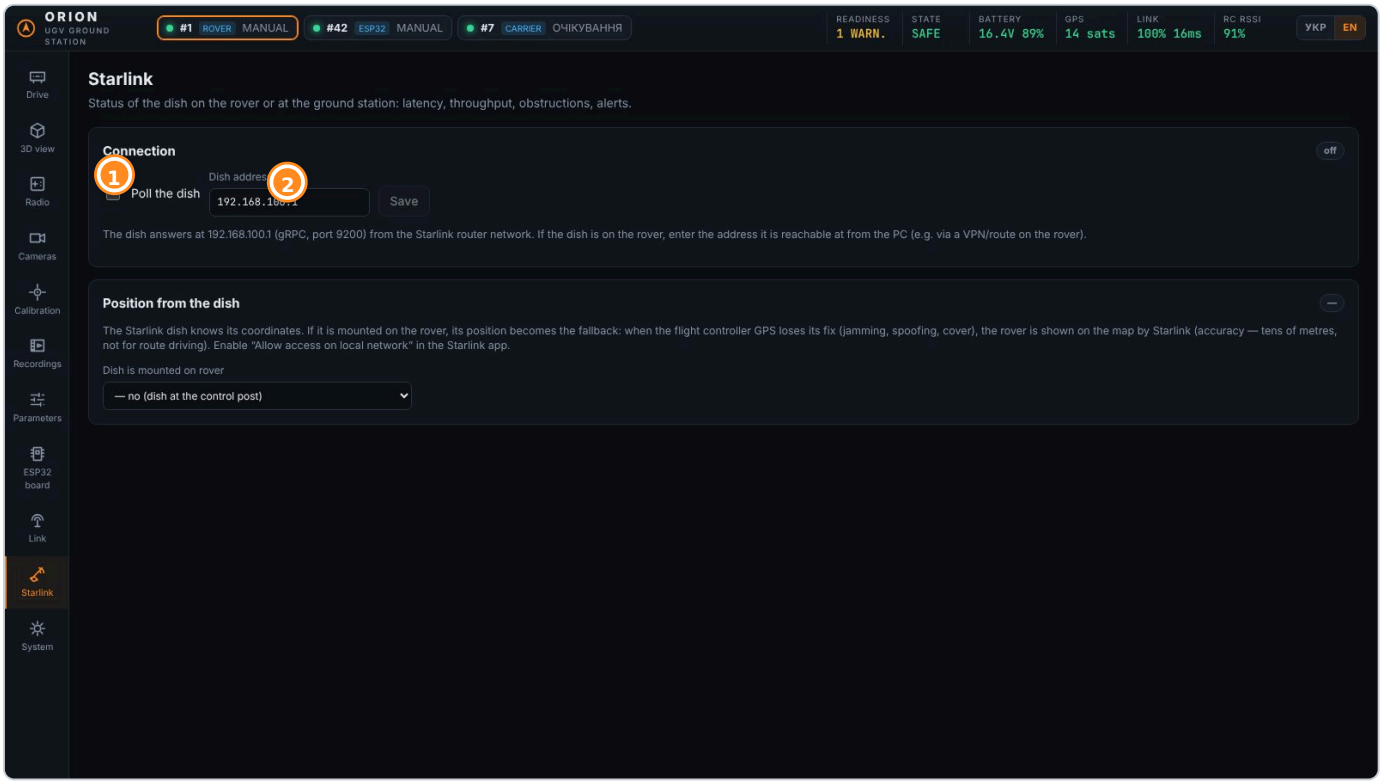



*MAVLink forwarding (“Link”, bottom of the page)*

1. **State:** how many frames were sent.
2. **Enable forwarding.**
3. **Comma-separated addresses:** `tcpin:5762` — TCP server (in Mission Planner: TCP, 127.0.0.1, 5762); `udp:127.0.0.1:14551` — send to a port (QGroundControl listens on 14550/14551); `udpin:14552` — listen on a port. Without an address tcpin/udpin accept only this PC; for other PCs use `tcpin:0.0.0.0:5762`.
4. **Allow other stations to control the vehicle** — then commands, parameters and missions from Mission Planner go to the vehicle. Without the checkbox other stations only watch.
5. **Each address:** how many stations are connected, or an error (e.g. port in use).

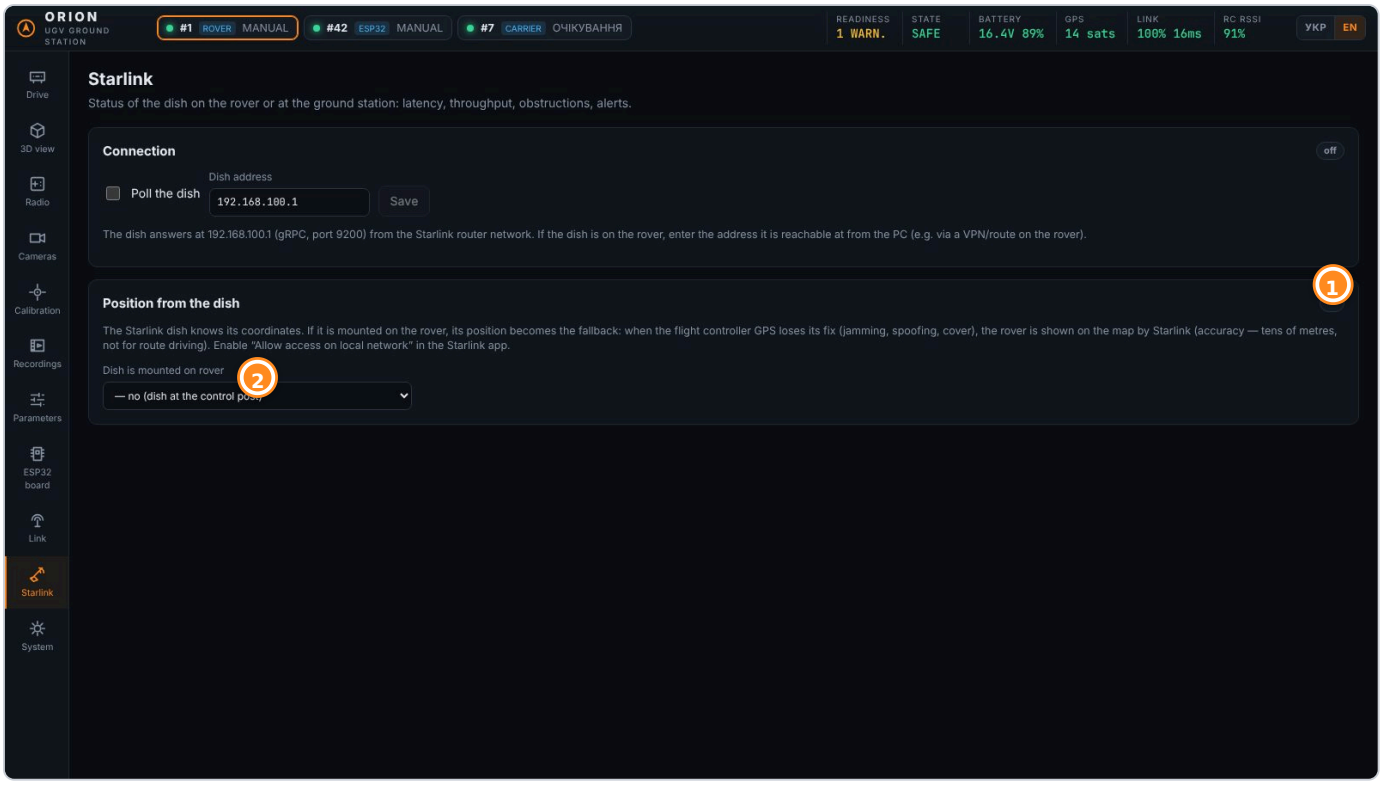
If you allow control, set a different station SYSID in Mission Planner than in ORION (255 by default), otherwise the flight controller will confuse the stations.

# 16. Starlink




*The "Starlink" section*

1. **Poll the dish** — turns on the Starlink status: latency, speed, obstructions, alerts.
2. **Dish address:** 192.168.100.1 by default. If the dish is on the rover, enter the address through which the PC can reach it (via VPN / a route on the rover).



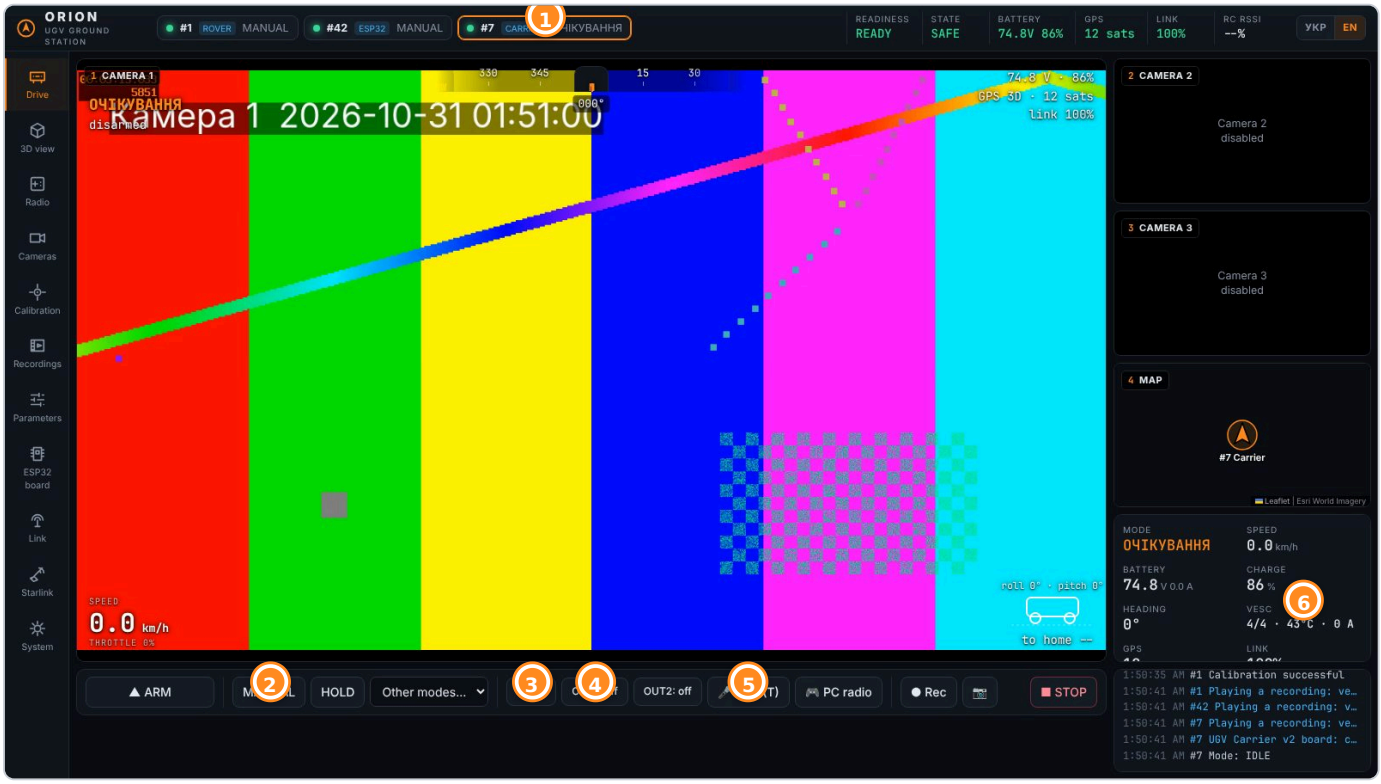

*Starlink → “Position from the dish”*

1. **State:** accuracy (±m) and the source of the dish position, or the reason why there is no position.
2. **Dish is mounted on rover** — select the rover. Then, if the flight controller GPS loses its fix (jamming, spoofing, cover), the rover position on the map comes from Starlink: a dashed accuracy circle appears around the rover marker and the telemetry panel shows “Position from Starlink”.

“Allow access on local network” must be enabled in the Starlink app, otherwise the dish does not report its coordinates. Accuracy is tens of metres: for finding the rover, not for route driving.

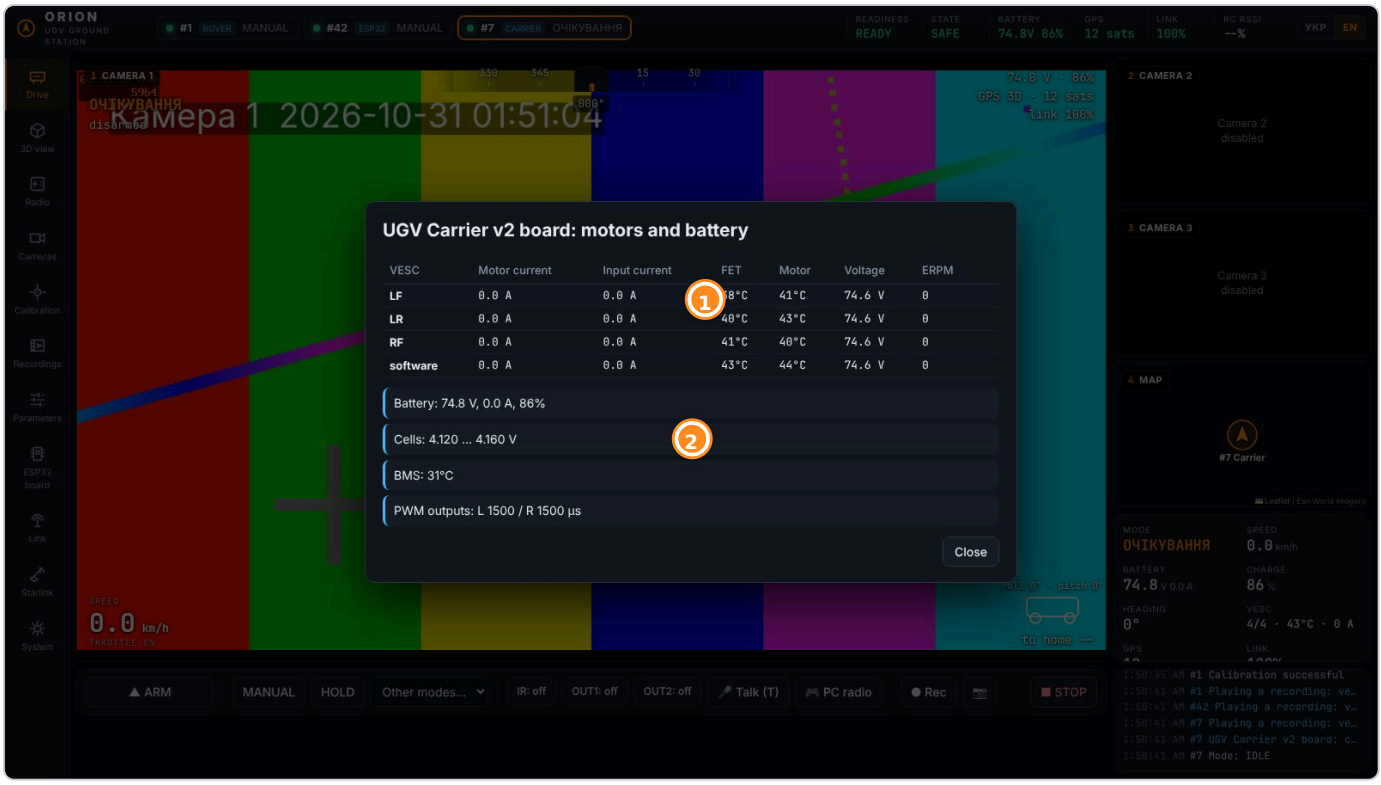
# 17. UGV Carrier v2 board

The UGV Carrier v2 board (ESP32-S3 with four VESC controllers) is detected automatically — the top bar shows a **CARRIER** tag next to the vehicle number. Nothing needs to be set up separately: the board sends telemetry to UDP 14550, like the ESP32 board.




*The “Drive” screen with a Carrier board*

1. **Carrier board** and its mode: IDLE, STOP, RADIO (the radio transmitter drives), PC (ORION drives), HOLD, FAILSAFE.
2. **MANUAL / HOLD** — hand control to the PC or stop the rover.
3. **IR** — IR lights.
4. **OUT1 / OUT2** — 12 V outputs on the PWR-OUT board.
5. **Talk (T)** — intercom: hold the button or the `T` key and speak into the PC microphone; the voice from the rover plays through the PC speakers. The link is half-duplex: when the person at the rover presses their button, the button shows “UGV talking…”.
6. **VESC** — how many controllers are online, the highest temperature and the total current. Yellow means overheating or a power limit. Click to open the details.



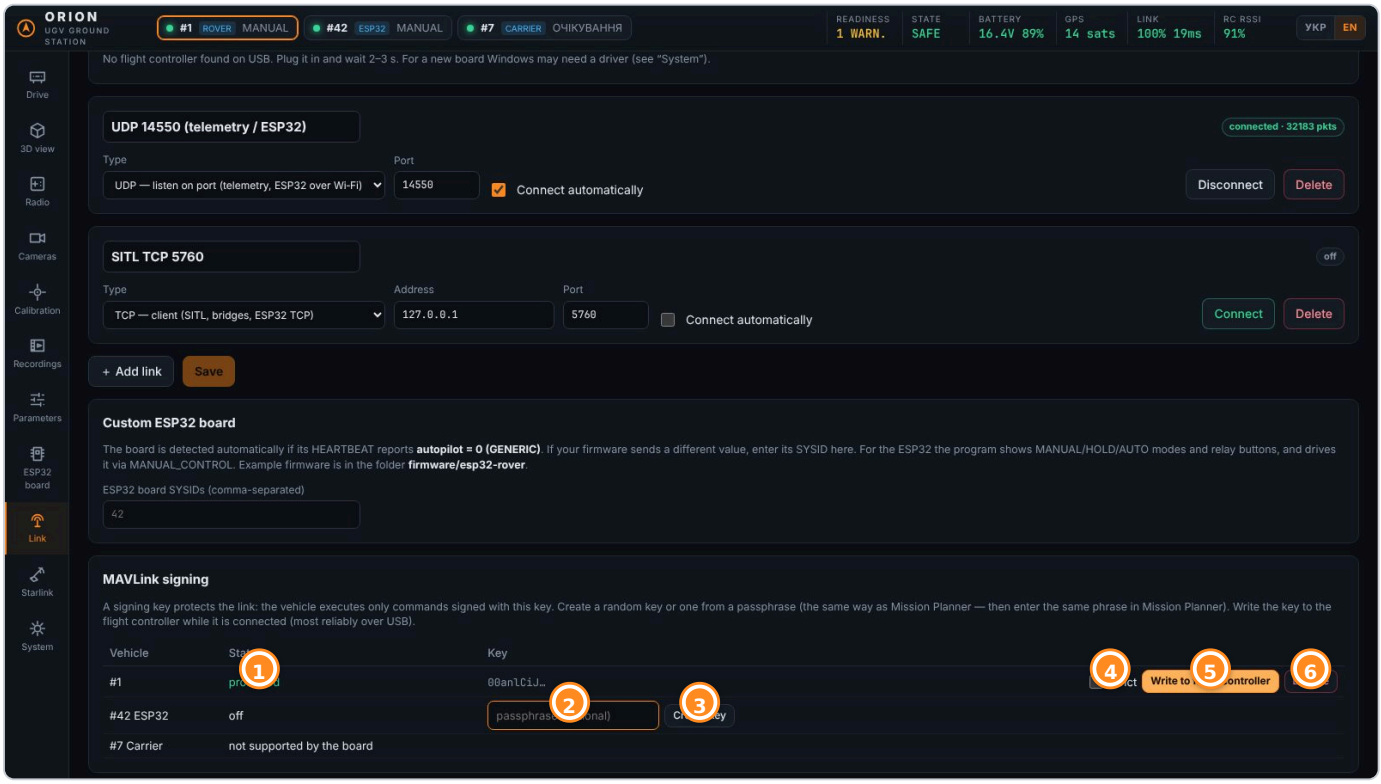

*VESC and battery details*

1. **Each VESC** (LF, LR, RF, RR — left/right, front/rear): motor and input current, transistor (FET) and motor temperature, voltage, RPM. “no link” — the controller does not respond.
2. **Battery:** voltage, current, charge; minimum and maximum cell voltage, BMS temperature, power limit (if the board enabled it) and PWM outputs.

The intercom works over UDP 14560 directly with the board. On the first press Windows asks for microphone permission — allow it. If the button shows ⚠, the board does not respond on that port.

# 18. MAVLink signing: protecting the link

Without signing, anyone with access to the link (the same radio modem, Wi-Fi, VPN) can control the vehicle. MAVLink signing (as in Mission Planner) adds to every frame a tag that can only be created with the key: the flight controller executes only signed commands.



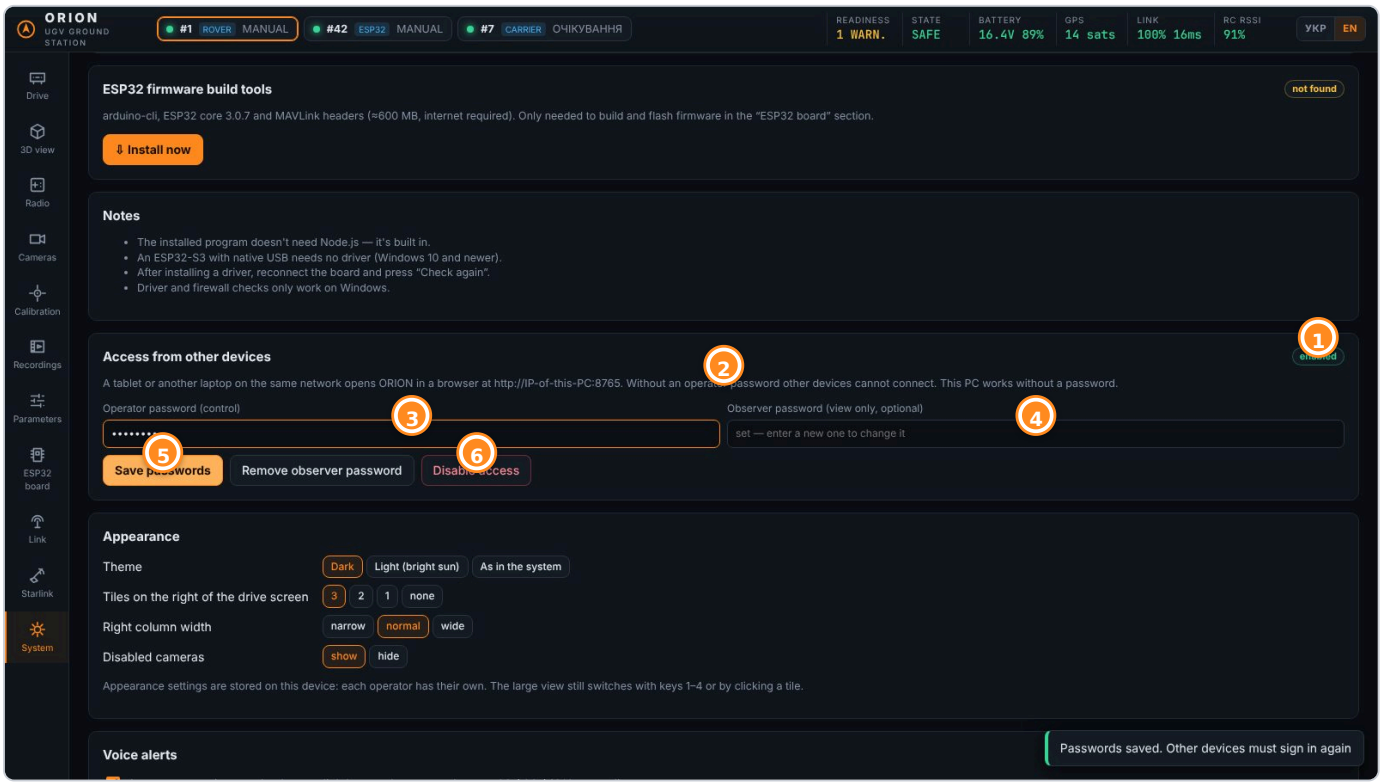

*"Link" → "MAVLink signing"*

1. **State:** disabled / "vehicle does not sign — write the key" / "some frames unsigned" / **protected**. Also how many foreign frames were rejected.
2. **Passphrase** — optional. With the same phrase Mission Planner derives the same key. Without a phrase a random key is created.
3. **Create key.**
4. **Strict** — reject vehicle frames without a valid signature (protection against spoofed telemetry).
5. **Write to flight controller** — the program sends the key and waits until the flight controller starts signing. Most reliable over USB.
6. **Disable** — send a zero key to the flight controller and return to an unsigned link.

If the key on the PC is lost and the flight controller already signs — connect the flight controller over USB (over USB ArduPilot also accepts unsigned commands) and disable signing or write a new key. ESP32 and Carrier boards do not support signing. A "MAVLink signing" item is also in "Readiness" — as a reminder.

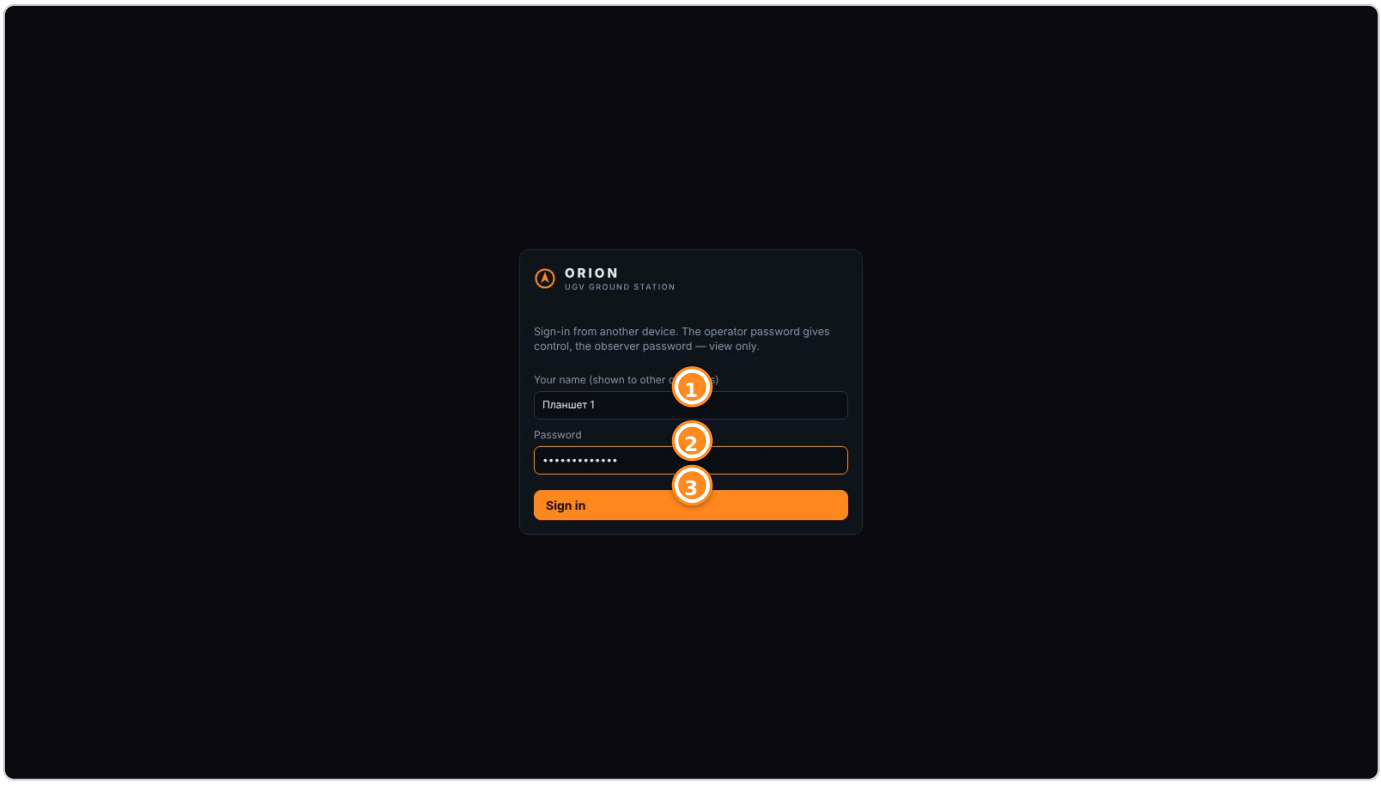
# 19. Access from other devices and multiple operators

ORION on the PC is also a web server: a tablet, phone or another laptop on the same network (control-post Wi-Fi, VPN) opens the program in a browser at `http://PC-IP:8765`. So that not just anyone with network access can drive the rover, sign-in from other devices is password-protected. The PC running ORION itself works without a password.




*"System" → "Access from other devices"*

1. **State:** "disabled" — other devices cannot connect at all (the default after installation); "enabled" — after signing in with a password.
2. **Address** to open on the tablet. In Windows the PC's IP is shown in Settings → Network → connection properties (or with the `ipconfig` command).
3. **Operator password** — full control: modes, ARM, PC transmitter, settings. At least 6 characters.
4. **Observer password** (optional) — view only: cameras, map, telemetry, 3D view. Handy for a commander or for training.
5. **Save passwords.** A new operator password signs out all devices — they must sign in again; a new observer password signs out observers only. Here too — "Sign out all devices" if a tablet is lost.
6. **Disable access** — remove the passwords and disconnect everyone connected from other devices.



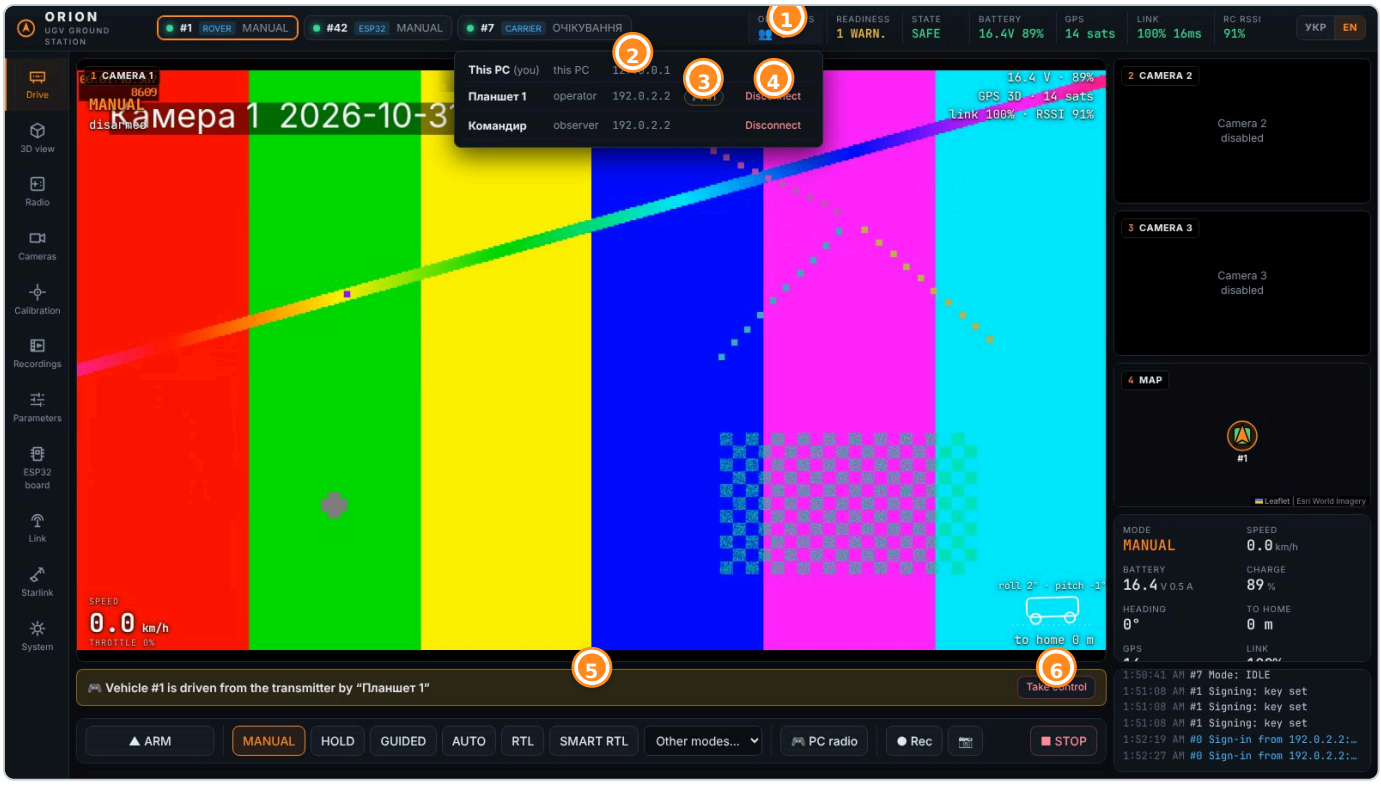

*Sign-in on a tablet*

1. **Your name** — this is how other operators see you and how your commands are signed in the event log.
2. **Password** of the operator or the observer.
3. **Sign in.** The browser remembers the sign-in for 30 days. After 5 wrong passwords in a row, sign-in from that address is blocked for 30 s.

Passwords are stored on the PC only as a hash (a separate access.json file); they are not visible in the settings and cannot be read from a tablet. Passwords can be changed only on the PC running ORION. The program also does not accept commands from foreign websites opened in the browser — only from ORION's own pages.

## Multiple operators

Several people can be connected at once: for example, one drives the rover, another watches the cameras from another PC, the commander observes from a tablet. Each one selects the vehicle in the top bar for themselves — this does not switch the vehicle for others.



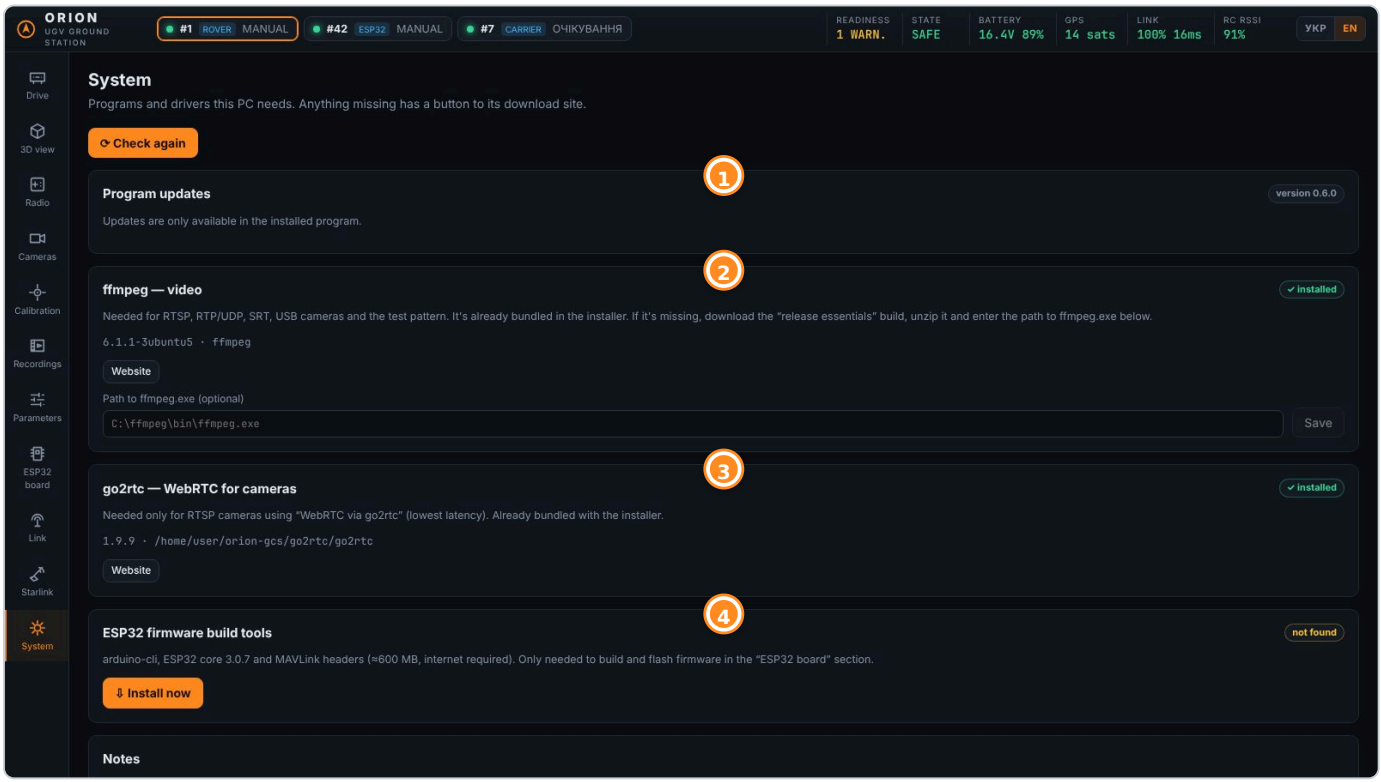

*Who is connected and who is in control*

1. **Operators** — how many clients are connected (appears when there is more than one). Click to see the list.
2. **List:** name, role (this PC / operator / observer) and address of each client.
3. 🎮 **#1** — this operator is driving vehicle #1 with the PC transmitter right now.
4. **Disconnect** — sign the device out (only on the PC running ORION).
5. **Who is in control:** only one operator at a time can drive a vehicle with the PC transmitter. While they send commands, the transmitters of others have no effect — they see this panel.
6. **Take control** — that operator's PC transmitter turns off and control passes to you. After 3 s without commands from the holder, control is released automatically.

- Mode buttons, ARM/DISARM and HOLD are available to all operators (so that anyone can stop the rover). Every such command is written to the event log with the name of whoever gave it: “Command from “Tablet 1”: HOLD”.
- An observer sees only “Drive” and “3D view”; the button panel is replaced by “View only”. MAVLink signing keys are not sent to observers.
- The Carrier intercom — operators only. **Only on the PC running ORION** (on a tablet these settings are visible but locked and marked 🔒): links, the ffmpeg path, the recordings folder, forwarding to Mission Planner, MAVLink signing keys, Starlink, the custom map server, the ESP32 board and its firmware, program updates, passwords. This way the operator password gives control of the rover, but not access to the PC itself.

# 20. System: dependencies, updates

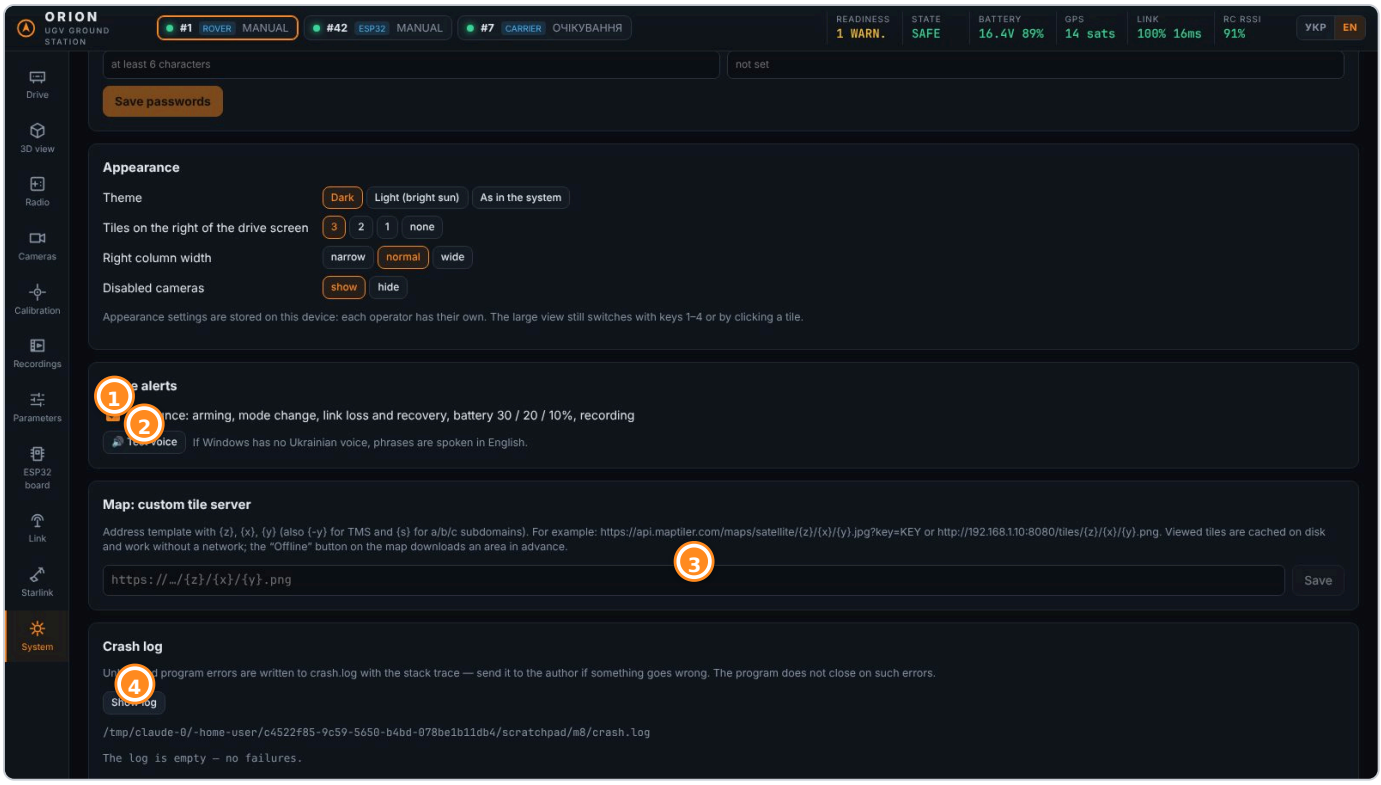
Here you can see what is missing on this PC, with download buttons. The **“Check again”** button at the top repeats the check (for example, after installing a driver).




*The “System” section*

1. **Program updates:** current version, checking and installing updates.
2. **ffmpeg — video.** Bundled with the installer. If you need a different build, enter the path to ffmpeg.exe.
3. **go2rtc — WebRTC for cameras** (section 12). Also already in the installer.
4. **ESP32 firmware build tools** — needed only for the “ESP32 board” section.

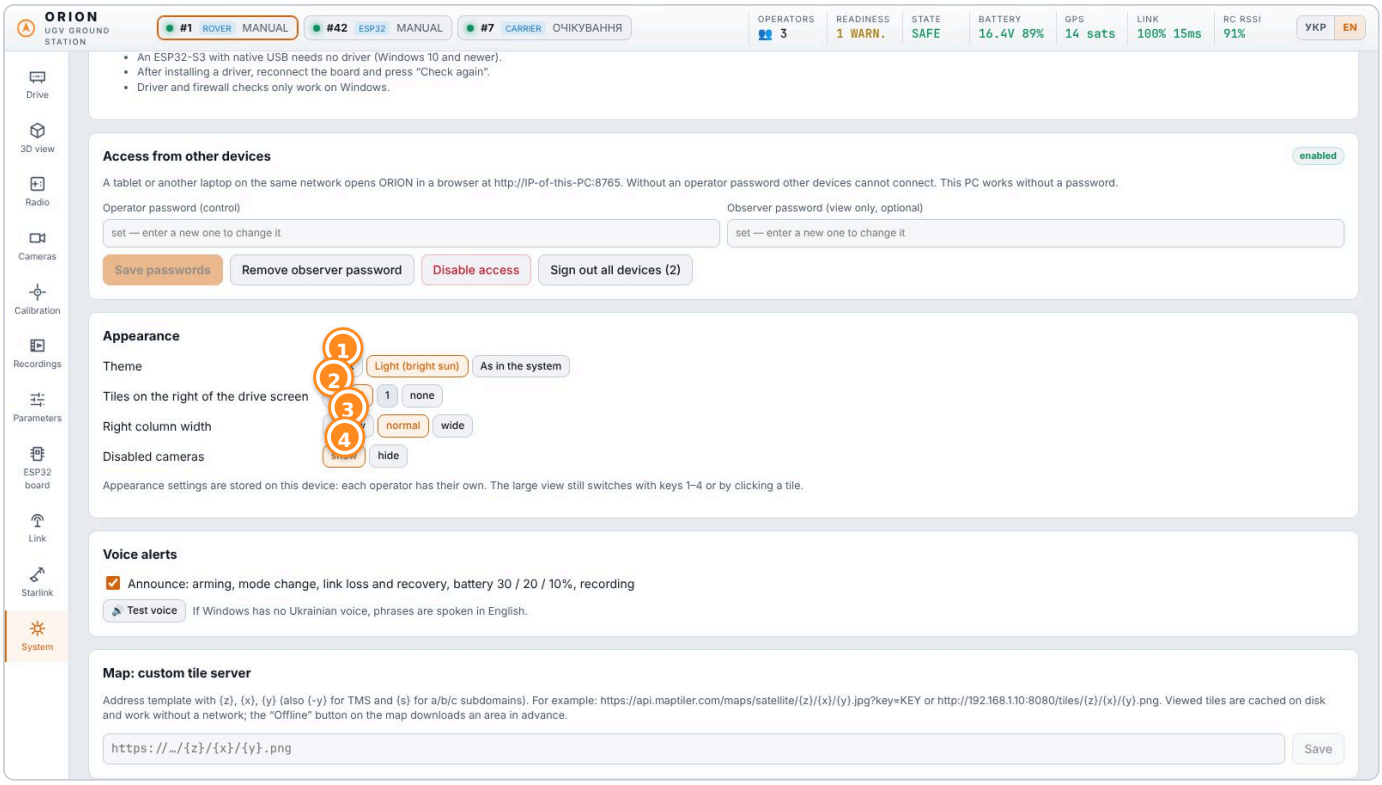
Further down the page — **Notes** (drivers for the CP210x, CH340, FTDI USB adapters also appear here if Windows lacks them, with a button to the manufacturer's site), access from other devices (section 19), appearance, voice alerts, custom map server, crash log and **About** (version, author, PDF manual).




*The “System” section: voice, custom map server, crash log*

1. **Voice alerts** — the program announces arming, mode changes, link loss and recovery, battery 30 / 20 / 10% and the start of recording. Handy when your eyes are on the video.
2. **Test voice.** If Windows has no Ukrainian voice, phrases are spoken in English (a Ukrainian voice is added in Windows Settings → Time & language → Speech).
3. **Custom map tile server** — an address template with {z}, {x}, {y}: MapTiler with your key, your own tileserver or a unit's offline server on the local network. After saving, a “Custom” layer appears on the map. Tiles are cached the same way and work without a network.
4. **Crash log** — unhandled program errors with details (crash.log). The program does not close on such errors; send the file to the author.

## Appearance and the light theme




*"System" → "Appearance" (light theme enabled here)*

1. **Theme:** dark, light — for bright sun when a dark screen is hard to see, or "as in the system" (Windows switches by day and evening).
2. **Tiles on the right** of the drive screen: 3, 2, 1 or none — then the large view is wider. Hidden cameras still open in the large view with keys 1–4.
3. **Right column width:** narrow — more room for the large view, wide — larger tiles and telemetry.
4. **Disabled cameras:** hide their tiles so they do not take up space.




*Light theme and two tiles on the right*

**Resizing with the mouse.** On the drive screen you can drag the borders: between the large view and the right column (column width), the bottom edge of each tile on the right (its height) and the top edge of the event log (its height). A border lights up orange when you hover over it. A tile grows into the telemetry space; to make it even larger, shrink another tile first. Sizes are saved automatically as soon as you release the mouse. Double-click a border for the default size; the “Reset sizes” button in “Appearance” restores everything. On a tablet, drag the borders with a finger.

Appearance is stored separately on each device: a tablet can have the light theme and one tile while the PC stays as usual.

> If something needed right now is missing (for example, a driver for a connected flight controller), a red reminder with a link to this section appears in the top bar.

# 21. Troubleshooting

| Problem | What to do |
|---|---|
| **The flight controller does not appear over USB** | Reconnect the cable (a data cable is required, not a charge-only one) and wait 2–3 s. Look at the “System” section — a driver may be needed. Close Mission Planner/QGC — they may hold the COM port. |
| **No telemetry over UDP / Wi-Fi** | Check that the PC is on the same network as the board. If the program was not installed with the installer, allow it in Windows Firewall. Through a router/VPN, enter the PC's IP on the rover. |
| **The rover does not arm** | The reason is in the event log (PreArm: ...). Most often — no GPS, accelerometer/compass not calibrated, low voltage. For a tabletop check — “Bench” → “Arm on the bench”. |
| **The sticks move but the wheels do not** | The rover must be armed and in MANUAL. In “3D view” check “Radio input” and “Output to controllers”. Check the output functions (step 1 of the radio wizard). |
| **The rover spins in place instead of driving straight (skid steering)** | The sides are swapped or one side needs reversing: swap the left/right side outputs in step 1 or set SERVOx_REVERSED in “Parameters”. |
| **The USB transmitter is not visible** | USB Joystick (HID) must be selected on the transmitter; move a stick. Other programs (simulators) may grab the joystick. |
| **The “Turn on PC control” button is inactive** | Press “Save profile”; a rover must be selected. The reason is shown next to the button. |
| **A camera shows nothing** | Check the address in VLC. Try “RTSP over TCP” or “Transcode”. Check ffmpeg in the “System” section. |
| **WebRTC via go2rtc — black screen** | The camera probably sends H.265: set H.264 in the camera or enable “Transcode”. The reason is shown on the camera tile. |
| **After writing the signing key the vehicle does not obey** | The key in the flight controller differs. Connect over USB and press “Write to flight controller” again, or “Disable”. |
| **The Carrier intercom is silent** | Allow the microphone for ORION in Windows (Settings → Privacy → Microphone). The firewall must not block UDP port 14560. |
| **The map is empty** | No internet — save tiles in advance with the “Offline” button. |
| **“Readiness” says the flight controller accepts control from another SYSID** | Press “Fix” next to that item or change MAV_GCS_SYSID (SYSID_MYGCS) in “Parameters” to 255. Otherwise PC transmitter commands are rejected. |
| **A recorded video does not open** | Open the .mp4 file in VLC. VLC picks up the .srt telemetry subtitles by itself if they lie next to it with the same name. |
| **Mission Planner does not see the vehicle through forwarding** | Check that forwarding is enabled and that Mission Planner uses TCP, 127.0.0.1, 5762 (or the port you set). The port must not be used by another program — the error is shown in the table under the addresses. |
| **The ESP32 board does not flash** | Hold the BOOT button as flashing starts; choose the correct COM port; install the CP210x/CH340 driver. |

| | |
|---|---|
| **A tablet cannot open ORION** | Set an operator password in “System” → “Access from other devices”, use the PC's IP address (not localhost) and allow the program in Windows Firewall. |

Questions and suggestions — to the author of the program, Anton Vasylenko.